

MI-Workbench: an autonomous multi-agent platform for mechanistic interpretability of biological foundation models

Ihor Kendiukhov^{1*}

¹ ~~Department of Computer Science~~[Institute of Medical Genetics and Applied Genomics](#), University of ~~Tuebingen, Tuebingen~~[Tübingen, Tübingen](#), Germany

* ~~ihor.kendiukhov@student.uni-tuebingen.de~~[gmail.com](mailto:ihor.kendiukhov@gmail.com)

Abstract

Mechanistic interpretability (MI) [analyses](#) of biological foundation ~~models—such as single-cell transformers trained on gene expression data—requires iterative cycles of hypothesis formation, experimental execution, statistical evaluation, and adversarial critique. These cycles are labor-intensive, error-prone, and difficult to reproduce~~[models are easy to get wrong: apparent regulatory signals can reflect co-expression, expression level or reference-network structure, and reviewing such analyses is laborious](#). We present MI-Workbench, an open-source platform that ~~coordinates a team of runs critique-and-revision loops on MI analyses with large language model (LLM) agents through a single, inspectable orchestrator to carry out MI investigations. Workflows are declared in a graph-based domain-specific language (DSL) whose nodes are agent roles—executor, a configurable—~~[Through an inspectable orchestrator, an executor performs and revises an analysis, optionally running its code in a sandbox, while a](#) panel of reviewer lenses (~~rigour~~[rigor](#), adversarial, biological plausibility) ~~, idea generator, and knowledge extractor—and each agent is reached through a provider-agnostic adapter layer supporting Claude Code, OpenAI Codex, and Google Gemini. Key mechanisms include: (i) a consensus reviewer that runs a reviewer panel concurrently and merges their critiques by Jaccard deduplication with agreement-based severity escalation and an optional advancement gate; (ii) structured,~~

artifact-aware feedback that maps critiques to specific artifacts and sections for targeted revision; (iii) optional sandboxed execution of the analysis code an agent emits, so reviewers critique executed results rather than an untested plan; critiques it. We built a benchmark of 12 flawed analysis write-ups of single-cell foundation models (48 planted flaws) and 6 clean controls, and (iv) an output-cleaning pipeline, automated convergence detection, and reproducibility packaging. We characterise the orchestrator with deterministic ablation and load experiments—larger reviewer panels surface strictly more distinct, complementary critiques, and the coordinator sustains concurrent runs with graceful tail-latency growth—and corroborate the panel with a live Gemini backend. A evaluated 1,296 reviewer calls from three OpenAI model configurations with two judges (flaw-detection agreement $\kappa = 0.84$). Three reviewer calls found more flaws than one (90–91% vs 81–86%), but three distinct lenses found no more than one lens sampled three times; distinct lenses mainly reduced noise and output tokens. Reviewers caught most statistical, confounding, causal and biological flaws, including the false premise that attention is symmetric, but only about one in five false statements about the analyzed model. Word-overlap merging of duplicate critiques failed at an uncalibrated threshold (F1 0.02) but nearly matched LLM adjudication once calibrated (held-out 0.80 vs 0.82). Real revision loops fixed most planted flaws in one revision but never satisfied the panel, so the default is a fixed budget. In an executed case study on whether Geneformer attention heads encode causal gene regulation shows the platform identifying key confounders (co-expression bias, rank-value encoding artifacts, attention symmetry) and refining analyses through structured feedback without human intervention. The system comprises roughly 9,200 lines of backend Python with an accompanying suite of over 370 tests, Geneformer attention and Tabula Sapiens data, every reported number traced to executed code, and supports real-time monitoring via WebSocket streaming. the panel identified at least 25 analysis errors beyond execution failures. Two of three runs agreed with an independent analysis that attention carries no signal about curated TRRUST edges beyond co-expression, rank and hub structure; runs also showed scope creep and hard-coded verdicts. MI-Workbench makes such analyses executable and auditable, but its conclusions require human acceptance.

Introduction

The rapid development of biological foundation models—including Single-cell foundation models such as scGPT [1], Geneformer [2], and scBERT [3]—has raised fundamental questions about what these models learn and how their internal representations relate to known biology. Mechanistic are increasingly used to make claims about gene regulation, and mechanistic interpretability (MI), the discipline of the reverse-engineering the computational mechanisms within of the computations inside neural networks [4–6], offers a principled framework for answering these questions. However, applying MI to biological models presents unique challenges: —is a natural way to test what these models have learned. MI analyses of biological models are, however, unusually easy to get wrong. Attention or embedding similarity between two genes can reflect co-expression, expression level or tokenization position rather than regulation [7]; curated reference networks are dominated by well-studied hub genes, which inflates recovery metrics; analyses that scan every head and layer invite uncorrected multiple comparisons; and attention weights are not faithful explanations of model computation in general [8–10]. Systematic evaluations of single-cell foundation models have repeatedly found that apparent signals shrink once such controls are applied [11–13].

1. **Confounding structures.** Gene co-expression, regulatory network degree distributions, and expression magnitude biases create complex confound structures that require carefully controlled null models [7].
2. **Statistical rigor.** Claims about learned regulatory mechanisms require multiple testing correction, effect size reporting, confidence intervals, and proper null model comparison—standards often missing from initial analyses.
3. **Iterative refinement.** MI investigations are inherently cyclical: an initial analysis reveals confounders, which motivate new controls, which reveal subtler effects, and so on.
4. **Reproducibility.** Complex computational pipelines with many degrees of freedom are difficult to reproduce without meticulous documentation of parameters, data versions, and analytical choices.

Catching these problems is the job of critical review. A single LLM prompted once to analyze a model reports its results without that scrutiny; LLMs can generate fluent but unsupported content [14], and the manuscripts of an automated research agent have contained incorrect or hallucinated numerical results even though the agent executed code [15]. LLM agent systems now automate large parts of the research cycle [16–19], including single-cell data analysis [20, 21], and several include an evaluator or critic agent. Yet it is rarely measured which errors such reviewers actually catch in analyses whose flaws are known, whether several specialized reviewers catch more than one reviewer given the same budget [22, 23], how their overlapping critiques should be merged, or when a review loop should stop.

~~While individual LLMs have demonstrated impressive capability for scientific reasoning [?, 30], a single agent prompted once lacks the adversarial pressure and structured quality control that rigorous research demands. A human researcher instead cycles between the roles of executor (performing analysis), reviewer (critiquing methodology~~MI-Workbench is an open-source platform for running such critique-and-revision loops on MI analyses of biological foundation models. A single orchestrator executes an explicit workflow graph whose nodes are LLM agent roles—an executor that performs and revises the analysis, a panel of reviewer lenses (rigor, adversarial, biological plausibility), and ~~devil’s advocate (searching for confounders and alternative explanations). MI-Workbench assigns these roles to distinct LLM agents and coordinates them.~~

~~The system is best described as an *agentic orchestration platform* rather than a distributed~~optional generative and knowledge roles—reached through a common adapter interface for the Claude Code, OpenAI Codex and Google Gemini command-line interfaces (CLIs); all evaluations reported here used OpenAI models through the Codex adapter. The platform is multi-agent in the sense that several role-specialized LLM agents, each with its own prompt and responsibility, act on the same analysis within every cycle, and autonomous in the sense that, once a task, workflow and stopping rule are set, a loop runs to completion without human intervention. It is not a multi-agent system in the classical sense: agents do not run ~~autonomous, asynchronous control loops or exchange messages through an agent communication language. A single orchestrator sequences them according to an~~

~~explicit-independent control loops, and the sequence of agent calls follows from the~~
~~workflow graph. This centralization is a deliberate design choice for scientific quality~~
~~assurance—it makes every run deterministic in its control flow, inspectable, auditable,~~
~~and reproducible—and it does not preclude concurrency where it matters: the~~
~~reviewer panel is evaluated in parallel~~ deliberate: every agent call passes through one
 coordinator that records it, so each run can be inspected and re-run with its recorded
 prompts and settings, although agent outputs vary between re-runs because the CLIs
 use each provider’s default decoding. The reviewer panel still runs concurrently within
 an iteration. ~~We introduce MI-Workbench, a platform that automates this multi-role~~
~~research process. Our key-~~

Our contributions are:

- A ~~graph-based workflow DSL~~ that lets researchers declare multi-role research
workflow layer for MI critique: graph-defined loops with conditional execution,
~~concurrent reviewer fan-out, and configurable stop conditions, validated before~~
~~execution.~~
- ~~A consensus reviewer mechanism~~ that runs a configurable panel of critic
~~lenses concurrently and merges their findings through deduplication, steps, a~~
consensus-review step with a machine-readable critique contract, configurable
merging of duplicate critiques (lexical, term frequency-inverse document
frequency [TF-IDF], embedding or LLM adjudication), agreement-based severity
~~escalation, deterministic ranking, and an optional advancement gate~~ escalation
and an advancement gate, configurable stopping rules, and a provenance record
of the model, reasoning-effort setting, CLI version, tokens and output of every
agent call.
- ~~A structured feedback loop~~ that maps reviewer critiques to specific artifacts
~~and sections, enabling targeted revision rather than wholesale rewriting~~ Verified
execution: the executor’s analysis code is run in an isolated
backend—resource-limited subprocess, kernel-enforced macOS sandbox, or
container—and reviewers critique the executed results; execution establishes
where reported numbers come from, not whether the analysis is correct.

- ~~Sandboxed-verified execution~~ that optionally runs the analysis code an 93
 executor emits and folds the captured results back into review, so critique 94
 targets executed output rather than an untested plan. 95
- A ~~provider-agnostic adapter architecture~~ abstracting over Claude Code, 96
 OpenAI Codex, and Google Gemini CLI backends through a common interface, 97
 with an integrated ~~knowledge extraction~~ pipeline and end-to-end 98
~~reproducibility packaging~~planted-flaw benchmark of 18 MI analysis write-ups 99
 of single-cell foundation models (12 flawed write-ups with 48 flaws of 12 types 100
 and 6 clean controls), and a pre-specified, call-matched evaluation of single 101
 reviewers, self-ensembles and the three-lens panel with real LLMs. 102
- ~~A quantitative evaluation~~ of the orchestrator—panel-size ablations, 103
 reviewer-lens complementarity, convergence behaviour, and concurrent-run load 104
 benchmarks—alongside a domain case study Data-driven calibration of the 105
 merging method and of the stopping rule, using the benchmark ground truth 106
 and recorded revision loops. 107
- An executed case study on Geneformer attention and real single-cell data, in 108
 which every number the agents report is traced to executed code and the agents’ 109
 conclusions are checked against an independent analysis of the same data. 110

Related work 111

Autonomous ~~scientific-research~~ agents. The vision of AI-driven scientific 112
 discovery has been explored through systems like ChemCrow [29], which chains LLM 113
 reasoning with chemistry tools, and Coscientist [30], which demonstrated Several 114
 systems now automate large parts of the research cycle with role-specialized LLM 115
 agents. The AI Scientist chains ideation, literature search, code writing and execution, 116
 plotting, manuscript drafting and an automated LLM reviewer for machine-learning 117
 research [16, 24], and its successor replaces human-written code templates with an 118
 agentic tree search over experiments [25]; an independent evaluation found its outputs 119
 uneven [15]. Agent Laboratory takes a human-supplied idea through literature review, 120
 experimentation and report writing with specialist agents and optional human 121

feedback at each stage [17]. The AI co-scientist generates, critiques and ranks hypotheses in a tournament of agents [18, 26]; the Virtual Lab organizes a principal-investigator agent, specialist agents and a scientific-critic agent into structured meetings [19]; and Robin and Kosmos couple literature agents with data-analysis agents over long autonomous runs [27, 28]. Earlier systems demonstrated tool-augmented LLM reasoning and autonomous experimental design in chemistry. ~~SciAgents [31] explored [29, 30] and multi-agent scientific reasoning using graph-based ontologies. These systems focus primarily on wet-lab experimental design + reasoning over scientific knowledge graphs [31].~~

Agents for biological data analysis. Closer to our domain, CellAgent assigns planner, executor and evaluator roles to LLM agents that run single-cell and spatial-transcriptomics analyses through a curated toolkit, with an evaluator-driven self-reflection loop that triggers re-runs [20, 32]. CellVoyager proposes and tests hypotheses on single-cell data in notebooks [21]; Biomni combines retrieval-augmented planning with code execution across biomedical tasks [33]; AutoBA automates multi-omic pipelines with code generation and repair [34]; BioDiscoveryAgent designs genetic-perturbation experiments with a critic agent and is benchmarked against Bayesian-optimization baselines [35]; CRISPR-GPT plans gene-editing experiments [36]; and GeneAgent and CASSIA add self-verification and validator agents to gene-set and cell-type annotation [37, 38]. These systems combine specialist agents, iterative evaluation and executed analyses, as MI-Workbench ~~targets the distinct challenge of computational interpretability with its emphasis on statistical rigor and confound control~~ does; they target data analysis or experimental design rather than the critique of interpretability claims about a model.

~~Multi-agent LLM frameworks~~ **Automated interpretability.** Multi-agent debate and collaboration have been studied in frameworks such as CAMEL [48], AutoGen [49], and ChatDev [50]. MetaGPT [51] introduced structured output schemas for software engineering agents. CrewAI and LangGraph provide general-purpose agent orchestration. Classical multi-agent systems emphasise autonomous agents with independent control loops that coordinate through message passing; Within interpretability, LLMs have been used to explain and score individual neurons [39] and sparse-autoencoder features at scale [40], and MAIA equips a

vision-language agent with interpretability tools and validates it against synthetic neurons with known ground truth [41]. Comparable feature-annotation pipelines exist for protein language models [42, 43]. For single-cell foundation models, interpretability and evaluation studies have moved well beyond individual case analyses: zero-shot and multi-task benchmarks [11, 12, 44], attention-based network benchmarks [45, 46], sparse-autoencoder analyses [47] and multi-architecture evaluations of attention-derived edge scores [13] apply fixed protocols across models, tissues and references. What these studies leave to the investigator is the step between an analysis and its acceptance: identifying confounders, unsupported claims and technical errors in a specific analysis, and revising it accordingly. That critique-and-revision step is the focus of MI-Workbench instead adopts a centrally orchestrated design in which a single engine sequences role-specialised LLM calls along an explicit graph. We make this trade-off deliberately: for automated scientific critique, a deterministic and inspectable control flow is worth more than agent autonomy, because it is what makes a run auditable and reproducible. MI-Workbench also differs from these frameworks in its domain specialization—the agent roles, prompt templates, artifact types, and quality criteria are designed for mechanistic interpretability research—while keeping the orchestration core domain-agnostic, so the same machinery drives other domains by swapping prompts.

Mechanistic interpretability **Multi-agent critique and its baselines.** The MI field has produced foundational work on circuits [4], induction heads [?], activation patching [?], and superposition [5]. Recent work has applied MI concepts to biological transformers: Geneformer [2] and seGPT [1] have been probed for gene regulatory network (GRN) inference capabilities. However, these analyses are typically manual, one-shot investigations. MI-Workbench provides the infrastructure to systematize and scale such investigations. Reviewer panels are one instance of multi-agent collaboration and debate [48–52]. MARG distributes paper reviewing over specialized agents and reduced the share of generic comments in a user study [53], and AgentReview simulates peer-review dynamics with LLM agents [54]. A consistent finding is that multi-agent gains shrink against budget-matched baselines: sampling one model several times and aggregating [55, 56], or a single strongly prompted agent [22], often matches debate [23], and failure analyses of multi-agent frameworks

report small gains on popular benchmarks [57]. LLM judges carry position, verbosity and self-preference biases [58, 59]. Grouping paraphrased outputs by meaning rather than shared words is the basis of semantic-uncertainty methods [14, 60] and of embedding-based similarity [61]. We therefore evaluate the reviewer panel against single-reviewer ensembles with the same number of calls, on items with planted ground truth, with two judges, and we calibrate critique merging against that ground truth.

~~Scientific workflow~~ **Workflow** systems. ~~Systems like~~ Galaxy [62], Nextflow [63] , and Snakemake [64] ~~manage computational biology pipelines. These tools orchestrate deterministic code execution;~~ orchestrate deterministic computational-biology pipelines, and general agent frameworks such as AutoGen [49] orchestrate LLM calls. MI-Workbench ~~orchestrates reasoning via LLM agents, where outputs are non-deterministic and require quality assessment.~~ sits between the two: its control flow is an explicit, validated graph, as in workflow systems, while its steps are stochastic LLM calls whose outputs must be critiqued rather than merely executed.

Positioning. Most components of MI-Workbench—role prompts, reviewer loops, code execution and artifact persistence—have precedents in the systems above. Its contribution is narrower: an inspectable orchestration layer specialized for critique of interpretability analyses; a critique-aggregation mechanism whose merging and stopping defaults are calibrated against planted-flaw ground truth rather than set by hand, and whose agreement-based escalation is evaluated against the same ground truth; verified execution with explicit confinement backends; and a planted-flaw benchmark that measures what automated reviewers actually catch in mechanistic-interpretability analyses of single-cell foundation models. Table 1 compares MI-Workbench with the most closely related systems. Its specialization for interpretability lies in the reviewer prompts and in the benchmark used to evaluate them; because every benchmark condition uses the MI-specific prompts, the benchmark measures what these prompts catch, not whether they catch more than a domain-agnostic reviewer prompt would.

Table 1. MI-Workbench and closely related agent systems.

System	Domain	Critique roles	Executes analysis code	How the critique step was evaluated
The AI Scientist [16, 24, 25]	Machine-learning research	One automated paper reviewer	Yes	Reviewer decisions compared with human conference decisions
Agent Laboratory [17]	Machine-learning research	Automated reviewer agents; optional human feedback	Yes	Automated review scores compared with human ratings
CellAgent [20, 32]	Single-cell and spatial data analysis	One evaluator driving self-reflection	Yes (curated toolkit)	End-to-end analysis quality and task completion
MARG [53]	Paper reviewing	Several specialized reviewer agents	No	User study of comment specificity
MI-Workbench	MI of single-cell foundation models	Three-lens panel with merging, escalation and stopping rules	Yes, in confined backends	Planted-flaw benchmark with two judges against single reviewers and self-ensembles with the same number of calls; merging and stopping calibrated on ground truth

Features as described in the cited publications.

Materials and methods

System overview

MI-Workbench ~~follows a layered architecture~~ is organized in four layers (Fig 1A): an interface layer (a command-line interface, a REST application programming interface (API) comprising: (i) a REST API layer built on FastAPI with WebSocket support for real-time monitoring; (ii) an orchestration engine that drives the multi-agent loop; (iii), and a WebSocket stream for monitoring); an orchestration layer that compiles a workflow graph and drives the agent loop; a provider-agnostic adapter layer that interfaces with LLM backends ; and (iv) persistence and artifact management via reaches LLM backends through their command-line interfaces; and a persistence layer (SQLite and a structured file system.

~~MI-Workbench architecture. The system comprises four layers: an interface layer (CLI, REST API, WebSocket), an orchestration layer (loop engine with state machine, convergence detection, feedback formatting, consensus review, and follow-up generation), a provider-agnostic adapter layer, and a persistence layer.~~

System architecture

The four layers follow a strict separation of concerns, each isolating one axis of change. The interface layer decouples how a run is launched (CLI, HTTP, or a monitoring socket) from what it does. The orchestration layer holds all artifact directory). All coordination logic—graph compilation, sequencing, review merging, convergence, and follow-up generation—in one place, which is what makes control flow auditable (a single reader can follow an entire run) and is the layer this paper evaluates. The adapter layer is a deliberate anti-corruption boundary: because every step sequencing, critique merging, stopping and verified execution—sits in the orchestration layer, so the control flow of a run can be followed in one place. Every backend is reached through one narrow run/smoke_test/is_available interface, the orchestration logic never depends on a vendor SDK, and new providers or a zero-cost mock are added without touching the engine. The persistence layer records every artifact, prompt, and run parameter so that any run can be reconstructed and packaged for reproduction. This is the standard layering of a service that must be simultaneously vendor-neutral and reproducible; the novelty is in the orchestration layer, not the tiering itself the same three-method interface, run(), smoke_test() and is_available(), so the engine does not depend on any vendor SDK and a deterministic mock backend can replace any provider in tests.

Workspace model

Fig 1. MI-Workbench architecture and review cycle. (A) The four layers. (B) One cycle of the consensus workflow: the executor's analysis code is run in a sandbox, the artifact and execution report are reviewed concurrently by three reviewer lenses, the lenses' critiques are parsed, merged, escalated on agreement and ranked, and the resulting required fixes are returned to the executor until a stopping condition holds.

A *workspace* ~~represents~~ is an isolated research context with its own configuration, provider settings, budget limits, and git integration mode. Each workspace contains one or more and budgets; a *run* executes one workflow on one task within a workspace. A run proceeds in runs iterations, where each run executes a loop definition against a specific research task. Runs produce versioned *artifacts* organized by iteration: mechanistic analysis (each of which is one step of the workflow: one executor call, or

one review or consensus step (a whole panel counts as one iteration). One cycle of the consensus workflow, an executor step followed by a panel step, therefore comprises two iterations. Each iteration writes its outputs to `runs/<run_id>/iter_NNNN/`: the step’s full output, the artifacts parsed from it (for example `MECH.md`), evaluation reports (and `EVAL.md`), experimental protocols (`XP.md`), methodology documentation (`METHOD.md`), and knowledge summaries (`KNOWLEDGE.md`).

Data model

The system uses Pydantic [71] for all data validation and serialization. Key models include for consensus steps each lens’s output and a machine-readable merge record, and for executed steps the full execution report. For every iteration the database and `RunState` `run_meta.json` (tracking run lifecycle, iterations, token usage, and cost), `IterationResult` (individual step outcomes with artifacts and feedback), `ReviewResult` (structured critique with severity levels and artifact references), and `LoopDefinition` (graph-based workflow specification). record the provider, model, reasoning-effort setting, CLI version, input, cached input and output tokens, duration, grade and, for the run, the reason it stopped. A reproducibility package bundles these files with the resolved prompts, the software environment and the git state. Further system details are in S1 Text.

Graph-based workflow DSL Workflow graphs

MI-Workbench defines research workflows as directed graphs where nodes represent agent roles and edges encode execution flow with conditions.

Loop definition

A loop definition consists of *nodes* and *edges* specified in YAML:

Edge conditions

Three condition types are supported: Workflows are directed graphs declared in YAML. Nodes are agent roles with a prompt template; edges carry one of three conditions: `always`: the edge is traversed every iteration. `every_k:N`: the edge is

~~traversed every~~; every_k:N, which runs the target in every N -th iteration (e.g., adversarial review every 3rd cycle). ~~on_flag:F~~: the edge is traversed only when flag F is set (e.g., ~~on_flag:high_risk_claims~~). cycle of the plan; and on_flag:F, which runs the target when the run flag F is set. Before execution the graph is validated (node references, condition syntax and stop-condition values) and compiled into an ordered plan that repeats until a stop condition holds. Nodes with the role consensus_merger absorb their incoming always edges into a single concurrent panel step. The consensus workflow, bundled as the reviewer_consensus preset and used in the stopping calibration, the case-study agent runs and the control-plane overhead measurements, is (node settings omitted):

Compilation

~~The DSL compiler performs a topological walk of the graph, validates structural integrity (no unreachable nodes, no duplicate IDs), and produces an ordered ExecutionPlan with a cycle length for infinite-loop repetition. Stop conditions include budget limits (tokens, cost, iterations), convergence detection, grade thresholds, and user-initiated cancellation.~~

```
nodes:
  - {id: executor, role: executor, prompt_ref: executor/mi_executor}
  - {id: reviewer, role: reviewer, prompt_ref: reviewer/mi_reviewer}
  - {id: adversarial_reviewer, role: adversarial_reviewer,
    prompt_ref: adversarial_reviewer/adversarial_reviewer}
  - {id: bio_plausibility_checker, role: bio_plausibility_checker,
    prompt_ref: biological_plausibility/bio_plausibility_checker}
  - {id: consensus_merger, role: consensus_merger}
edges:
  - {source: executor, target: reviewer, condition: always}
  - {source: executor, target: adversarial_reviewer, condition: always}
  - {source: executor, target: bio_plausibility_checker, condition: always}
  - {source: reviewer, target: consensus_merger, condition: always}
  - {source: adversarial_reviewer, target: consensus_merger, condition: always}
  - {source: bio_plausibility_checker, target: consensus_merger, condition: always}
  - {source: consensus_merger, target: executor, condition: always}
```

Built-in presets

Four presets are provided: `executor_reviewer` (standard executor-reviewer loop with periodic adversarial review), `reviewer_consensus` (three parallel reviewers (standard, adversarial, bundled: `executor_reviewer` (executor and rigor reviewer, with the adversarial reviewer in every third cycle), `reviewer_consensus` (the panel workflow above), `biological_plausibility`) with consensus merger), `research_followups` (executor-reviewer loop with automated `research_followups` (executor and a follow-up proposal generation and optional auto-dispatch), `example_custom` (advanced loop with `idea_generator`) and `example_custom` (conditional adversarial escalation, knowledge extraction, and graph stability detection, and loop-level stop conditions)).

Multi-agent orchestrationAgent roles and prompts

A single orchestrator drives every run as a synchronous loop over the compiled plan (Algorithm ??), issuing one agent step per iteration. This centralization is what makes a run reproducible and auditable: the sequence of agent calls for a given loop and task is fully determined by the graph, and the entire control flow lives in one module a reviewer can read end to end. Concurrency is introduced deliberately and locally—within a consensus step the reviewer panel is dispatched with `asyncio.gather`, and independent runs execute as separate asynchronous tasks (up to a configured ceiling)—so the orchestrator scales across concurrent investigations without sacrificing the determinism of any single one. We quantify both properties in the Results.

Nine roles are available (Table 2), each configured through a role-specific prompt template. A review loop needs only the executor and one or more reviewer lenses, and all evaluations in this paper use only these four roles; the five generative and knowledge roles are optional and were not evaluated. The three lenses emphasize different failure modes: the rigor lens checks claim validity, reproducibility, confounders, negative controls and biological plausibility; the adversarial lens probes seven threat models (p-hacking and forking paths, causal over-claiming, causal confusion, leakage and circularity, biological implausibility, statistical theater and inadequate null models); and the biological-plausibility lens checks tissue and cell-type

context, pathway consistency, expression plausibility and regulatory direction. The lenses share some items by design (plausibility appears in all three). A combined-checklist reviewer prompt contains every item of the three lens prompts verbatim; it is the strongest single-reviewer baseline in the evaluation.

Agent roles

MI-Workbench defines nine agent roles (Table 2) that fall into four groups. The **executor** performs analysis and emits artifacts. Three **reviewer lenses**—rigour, adversarial, and biological plausibility—critique that output; they are intentionally distinct rather than redundant, each trained by its prompt on a different failure mode (statistical rigour, adversarial confounds, and domain plausibility, respectively), and we show in the Results that they surface largely non-overlapping critiques. Two **generative** roles (idea generator, automator) propose follow-up experiments and reusable pipelines, and three **dissemination and knowledge** roles (knowledge extractor, publication manager, article publisher) structure findings and mine claims. Only the executor and the reviewer lenses are required for a review loop; the remaining roles are optional nodes a workflow may include. Because roles are graph nodes rather than hard-coded stages, a workflow toggles any role by including or omitting it, and the reviewer panel is resized simply by adding or removing lens nodes.

Agent roles and their research functions.

Table 2. **Agent roles.** The evaluations reported here use only the first four roles; the remaining five are optional and were not evaluated.

Role	Function
Executor	Performs and revises the analysis; with verified execution, writes one analysis script whose outputs are reported
Reviewer (rigor lens)	Claim validity, reproducibility, confounders, negative controls, biological plausibility
Adversarial reviewer	Seven threat models for common failure modes of interpretability analyses
Biological-plausibility checker	Tissue and cell-type context, pathway consistency, expression plausibility, regulatory direction
Idea generator	Prioritized follow-up proposals with value, feasibility and novelty scores
Knowledge extractor	Claims, uncertainty estimates and falsification tests (S1 Text)
Automator	Reusable pipelines from validated protocols
Publication manager	Manuscript planning and checklists
Article publisher	Summaries for communication

Execution loop

The core execution loop (Algorithm ??) drives the iterative research process. Prompts are assembled by role. Only executor steps receive revision framing: the reviewers’ feedback, the executor’s previous submission and, when execution is enabled, its execution report. Every other step receives the task as context, the iteration number, and the executor’s latest full output, with its execution report, under an “artifact under review” header.

Consensus review

A consensus step dispatches the panel concurrently over the current artifact and combines the lenses’ critiques in four stages (Fig 1B).

Parsing. Every reviewer prompt ends with a required machine-readable block: a fenced JSON object listing critiques, each with a **severity** (critical, high, medium, low or info), a **category**, a **description** and a **required fix**. The parser reads this block first, tolerating common JSON defects (trailing commas, single quotes, truncation), then falls back to YAML-like and bracketed formats, and counts each critique once. A lens whose call fails, or whose output contains no readable critique block, is marked failed rather than read as a clean review. A failed panel is retried once and otherwise stops the run with a recorded reason. A partial panel (some lenses usable) is retried once for its failed lenses; if it is still partial, it proceeds, but by default its grade does not enter stopping decisions.

Merging. Critiques from different lenses that describe the same problem are grouped. Four methods are implemented: word-level Jaccard similarity; TF-IDF cosine similarity over normalized, stop-word-filtered and stemmed tokens; sentence-embedding cosine similarity [61] (optional); and LLM adjudication, in which one additional call partitions the numbered critiques into groups that identify the same underlying problem. For the similarity-based methods, critiques are grouped by average-linkage agglomerative clustering at a threshold τ , which makes the result independent of input order. By default, critiques from the same reviewer call are not merged with each other, for every merge method: every lens prompt asks for one critique per distinct problem, so two items from one call are kept as separate required

fixes; this rule is configurable. An adjudicator answer must be a valid partition of the critique indices; otherwise it is retried once, and if it is still invalid, TF-IDF grouping is used. Based on the merge evaluation (Results), LLM adjudication is the default method, and the thresholds of the Jaccard and TF-IDF methods are set to $\tau = 0.1$, the upper (more conservative) end of the range selected by cross-validation (0.075–0.1); in-sample F1 differs by less than 0.001 between these two values.

Escalation. Each group takes the highest severity among its members. If at least k distinct lenses contributed to a group (default $k = 2$), its severity is raised by one level, capped at critical. A merged critique keeps the text and proposed fix of every member.

~~Algorithm ?? shows the ordinary single-agent step for clarity. Two step kinds specialise it. A *consensus* step (compiled from a workflow whose reviewers fan out to a merger node) replaces the single `RUNWITHRETRY` call with a concurrent dispatch of the whole reviewer panel followed by a merge (described below). When verified execution is enabled, an executor step additionally runs the analysis code it emitted and appends the captured results to~~ **Ranking and grading.** Groups are ordered by severity, an optional per-lens weight, the number of contributing lenses and the critique text, so the order never depends on dispatch or parsing order. A letter grade summarizes the merged list (two or more critical critiques give F; one critical or three or more high give D; one or more high give C; any medium gives B; otherwise A). Because LLM reviewers raise high-severity concerns about essentially every analysis (Results), the grade is reported as an ordinal summary for monitoring; it is not used as an absolute acceptance criterion by default.

Gate. An optional advancement gate keeps the loop revising while an unresolved critical critique remains; budgets and the iteration cap still bound the run, so a panel that never agrees terminates.

Feedback to the executor

Merged critiques are stratified by severity: critical and high critiques become required fixes, and medium and lower critiques become suggested improvements. Each entry lists the severity, category, any inferred artifact and section reference, the critique, the

reviewer's required fix and the ~~feedback before the reviewers see it~~ points of merged
duplicates. The executor is asked to make targeted revisions to the referenced artifacts
rather than rewrite them.

~~Stopping and iteration budget~~

~~A run continues until~~

Stopping

A run stops at the first of ~~four conditions holds: the iteration ceiling~~ (`: user`
cancellation; the iteration cap; the token or cost budget (input and output tokens and
cost as reported by the provider); a loop stop condition
(`maxgrade_iterationsat_least:G, default 50`)~~is reached; on flag:F~~; or the
stopping rule. The stopping rule evaluates, after a minimum number of iterations
(default 4, i.e. two executor-panel cycles of the consensus workflow), three signals over
a window of w observations (default 3): the merged grade is unchanged; no critical or
high critique was raised; and consecutive executor outputs have word-level Jaccard
similarity of at least 0.9. The combination of signals is configurable (any, all, k of n ,
with optional required signals). Failed panels never enter the signal history, and
partial panels do not by default. In addition, a revision budget limits the ~~token budget~~
(`default 500, 000`)~~or cost budget is exceeded; convergence is detected; or the user~~
~~cancels. These bounds cap the number of feedback rounds; a typical review loop~~
~~converges well before the ceiling~~ executor revisions after the initial submission; when it
is reached, the remaining review steps of the cycle run, so the final artifact is reviewed,
and the loop stops. Based on the stopping calibration (Results)~~—~~, the default is a
budget of four revisions with the adaptive rule disabled; the adaptive rule can be
enabled per run. The reason a run stopped is recorded.

~~State machine~~

~~Run lifecycle is managed by a finite state machine with states {Pending, Running,~~
~~Paused, Stopped, Completed, Failed} and well-defined transitions. Paused runs can be~~
~~resumed with full state reconstruction from the database and artifact filesystem. A~~

~~crash recovery module detects interrupted runs on server restart, transitioning
Running states with existing iterations to Paused for resumability.~~

Verified execution

~~Convergence detection~~

~~After a minimum of four iterations, three convergence signals are monitored over a
sliding window of w reviews (default $w=3$):~~

- ~~1. **Grade stability:** the reviewer grade has remained constant for w consecutive
reviews.~~
- ~~2. **Zero critical issues:** no Critical or High severity critiques for w reviews.~~
- ~~3. **Output similarity:** Jaccard word-level similarity between consecutive executor
outputs exceeds threshold τ (default 0.9) for w iterations.~~

~~Any single signal triggers loop termination, preventing waste of compute on converged
analyses. When the consensus advancement gate is enabled, convergence is suppressed
while any Critical critique remains unresolved, so the loop does not declare success
over an open critical issue~~

When verified execution is enabled, the executor is asked for
one self-contained Python block that reads only the data paths named in the task,
prints a summary of its results, writes `results.json` and reports no number that the
code does not compute. After each executor step the block is run, and the execution
report—exit status, standard output, the tail of standard error, and the files written,
including `results.json`—is appended to the artifact the reviewers see and returned
to the next executor iteration. With execution enabled, the executor’s own agent tools
are disabled by default, so every reported number has to come from the executed code.
Verification here concerns provenance: it ensures that reported numbers were
computed by the executed code, not that the code, its numbers or the conclusions
drawn from them are correct.

Execution is isolated by one of three backends. All three run `python -I` in a fresh
temporary directory with a minimal environment, standard input closed, output
capped and a wall-clock timeout. The `-I` flag only isolates the interpreter from
`PYTHON*` environment variables and the user’s site-packages; it is not a security

boundary. The `subprocess` backend adds CPU-time, file-size and open-file limits and, on Linux, an address-space limit, runs the code in a new session and kills the whole process group on timeout; it does not restrict network or file-system access. The `sandbox_exec` backend adds kernel-enforced confinement on macOS through a Seatbelt profile. The profile denies network access, permits writes only in the temporary directory, and limits reads to system and interpreter paths and to explicitly listed read-only data directories. Data paths that are too broad (the file-system root, the home directory or volume roots) are rejected. The `docker` backend runs the code in a container without network access, with a read-only root file system, memory, CPU and process limits, dropped capabilities and a non-root user. On macOS, whose kernel does not enforce address-space limits, only the `docker` backend limits memory. If the requested backend is unavailable, execution fails closed. Automated tests verify network denial, write confinement, read allow-listing, CPU limits and process-group termination for the `sandbox_exec` backend by running code under the sandbox; tests of the `docker` backend check only the container command it constructs, and this backend was not used in the experiments reported here.

Retry logic

~~Adapter calls are wrapped in exponential backoff retry with jitter~~

Adapters

The Claude Code, Codex and Gemini CLIs are wrapped as adapters, and a mock adapter returns deterministic, role-aware responses for tests. Each adapter forwards the model and, where the CLI supports it, the reasoning-effort setting (the Gemini CLI has no effort flag, so a requested effort is recorded as unsupported), sends the prompt on standard input, runs the CLI in its own process group and parses the CLI's JSON output for the answer, the token counts and, where reported, the cost. Reviewer calls run with the CLI's agent tools disabled (for Codex: a read-only sandbox, no shell, no web search, no plugins or external tool servers, and the user configuration file, `config.toml`, ignored). Transient errors (rate limits, timeouts, connection failures) trigger retry with delay $d_k = d_0 \cdot 2^k \cdot (1 + \epsilon)$, where

$\epsilon \sim \mathcal{U}(-0.25, 0.25)$. Permanent errors (authentication failures, invalid requests) fail immediately; network and overload errors are retried with exponential backoff; authentication, quota and configuration errors are not. Sampling temperature is not exposed by these CLIs, so runs use each provider’s default decoding; the model, effort setting and CLI version of every call are recorded.

Consensus review mechanism **Planted-flaw critique benchmark**

A central mechanism of MI-Workbench is the *consensus reviewer*, which replaces single-reviewer assessment with a panel and a defined aggregation rule. To measure what automated reviewers catch, we built a benchmark of analysis write-ups with known flaws (S1 File). Twelve scenarios cover common interpretability analyses of single-cell foundation models: attention-based network recovery in Geneformer and scGPT, linear probing, in-silico perturbation, sparse-autoencoder features, head ablation, activation patching, cross-tissue threshold transfer, age signals in embeddings, attention-derived hub genes, perturbation-response prediction and gene-embedding similarity. Each scenario has a flawed write-up of 750–1,100 words (question, data and model, methods, numerical results, interpretation, limitations) containing four planted flaws, one from each of four of the five families in Table 3; each of the twelve flaw types occurs exactly four times (48 flaws). Three of the four attention-mechanics (T1) flaws are the claim that dot-product attention is symmetric. Six scenarios also have a clean counterpart in which all four flaws are corrected and no instance of any flaw type is present. Flaws are stated naturally, are detectable from the text alone and never appear among the stated limitations. For every flaw the ground truth records the verbatim passage, a description and a strict detection criterion. The items and their ground truth were written with Claude Opus 5.5 (Anthropic), used interactively outside the platform (see Use of language models outside the evaluated system). A separate instance of the same model, working in a fresh context, checked them for verbatim quotes, unplanned flaws, factual errors in non-planted statements and internal numerical consistency. Corrections were made before the benchmark was frozen (SHA-256 manifest) and before any reviewer call. The authoring model therefore comes from a different provider than all reviewer,

judge and adjudicator models (OpenAI).

540

Panel

541

~~The panel is a configurable list of reviewer lenses declared in the workflow graph; the default MI panel has three—rigour, adversarial,~~

542

Table 3. Flaw taxonomy of the planted-flaw benchmark. Each type occurs in four flawed items.

Family	Type	Example of the planted error
Statistics	S1 multiple comparisons	significance claimed from uncorrected per-head p -values across 144 heads
	S2 effect-size neglect	a negligible effect with a tiny p -value presented as substantive
	S3 metric misuse	AUROC as the sole metric for edges at 0.3% density
Confounding	C1 expression or co-expression	gene-pair scores interpreted as regulation without co-expression or expression baselines
	C2 reference-degree bias	recovery against a hub-dominated reference without a degree-preserving null
	C3 leakage	donors shared between training and test, or selection on the test set
Causal logic	L1 causal over-claiming	correlational attention evidence described as the model’s mechanism
	L2 circular validation	validation evidence that is not independent of how the finding was obtained
Technical	T1 attention mechanics	“dot-product attention is symmetric, so $A_{ij} = A_{ji}$ ”
	T2 model or tokenization	wrong tokenization scheme, training objective or layer count for a named model
Biology	B1 context implausibility	a regulator claimed active in a cell type where it is not expressed
	B2 regulatory logic	reversed regulatory direction or a misattributed pathway

Type codes are those used in the benchmark files and results (S1 File); they are unrelated to the reviewer conditions C1–C5 and to the numbering of the Supporting Information items. AUROC, area under the receiver operating characteristic curve.

543

Evaluation protocol

544

Conditions. For every item, model configuration and repeat we made eight independent reviewer calls on the same input: three with the rigor lens, three with the combined-checklist reviewer, one with the adversarial lens and one with the biological-plausibility lens. Five conditions were built from these calls: C1, one rigor reviewer; C2, one combined-checklist reviewer; C3, three rigor samples; C4, three combined-checklist samples; and C5, the three-lens panel (one rigor, one adversarial and one biological-plausibility call). The conditions share calls: the call of C1 is one of the three calls of C3 and the rigor call of C5, and the call of C2 is one of the three calls of C4. Because a flaw counts as found if any call of a condition identifies it, C3

553

and biological plausibility—but a workflow may use fewer or more, C5 cannot have lower recall than C1 on any item, nor C4 than C2; these contrasts measure how much the additional calls add, whereas the call-matched contrasts C5–C3 and the lenses are domain-configurable through their prompts. Within a consensus step the panel is dispatched concurrently over the current executor artifact, so panel latency is that of the slowest lens rather than their sum.

Critique merging

The merge operates in four phases. **Phase 1 (parsing):** each lens’s free text is parsed into structured `ReviewCritique` objects using multi-format detection—bracket `(SEVERITYdescription)`, Markdown `(**Severity: X** -- description)`, and structured `(Severity: X, Category: Y, Description: Z)`. C5–C4 test whether distinct lenses add recall. Conditions C3–C5 use three calls each, so the panel is compared with a self-ensemble of a single lens (C3) and with a self-ensemble of the strongest single reviewer (C4) at the same number of calls; token counts are reported per condition. Reviewers received a fixed task statement and the artifact, with tools disabled. Severity outcomes (merged grade, severity score, severity-aware recall and high or critical critiques on clean items) used one merge setting for all conditions, fixed with the analysis plan before any reviewer call: lexical Jaccard merging at $\tau = 0.5$. This threshold merges few paraphrased duplicates (see Merging paraphrased critiques), so these outcomes approximate counts over unmerged critiques. The same outcomes with TF-IDF merging at $\tau = 0.075$, the threshold selected in 11 of the 12 cross-validation folds of the merge evaluation, are reported as a sensitivity analysis (S2 Table). **Phase 2 (deduplication):** critiques are grouped by Jaccard word-overlap similarity,

$$J(c_i, c_j) = \frac{|\text{words}(c_i) \cap \text{words}(c_j)|}{|\text{words}(c_i) \cup \text{words}(c_j)|},$$

with pairs satisfying $J(c_i, c_j) \geq \tau$ (default $\tau=0.5$) merged into one group.

Phase 3 (agreement-based escalation): Models. each group takes the maximum severity present; if at least k distinct lenses raised the group (the *consensus threshold*, default $k=2$), its severity is escalated one level (capped at Critical), encoding the intuition that inter-lens agreement signals importance. **Phase 4 (deterministic**

~~ranking): groups are ordered by severity, then by an optional per-role weight, then~~
~~by the number of contributing lenses, with the critique text as a final tie-break so the~~
~~ranking never depends on parse or dispatch order.~~ We evaluated three model
configurations from one provider (OpenAI) through the Codex CLI (version 0.145.0):
gpt-5.6-sol at medium reasoning effort, gpt-5.5 at medium effort and gpt-5.6-luna at
low effort, each with three repeats (1,296 reviewer calls; S1 Table). Codex prepends to
every prompt the built-in harness instructions of the installed CLI version and the
user-level instruction file (AGENTS.md in the Codex home directory), which the CLI
reads even when config.toml is ignored. The instruction file is a generic persona for
agentic software engineering of about 4kB: it asks the model to name inconsistencies
and unclear specifications instead of guessing, to point out problems rather than agree,
to prefer simple code and to avoid changes outside the task. It contains nothing about
biology, interpretability, the benchmark items or the review task. Both texts were
identical for all calls, conditions and model configurations, including judge, executor
and adjudication calls, and their SHA-256 hashes are recorded for every call.

~~Consensus definition, gating, and deadlock~~

~~“Consensus” here is thus an explicit aggregation—dedup, agreement-weighted severity,~~
~~and deterministic ranking—rather than a vote, and~~ **Judging.** For each item,
configuration and repeat, one judge call received the item text, its planted flaws with
their detection criteria, and the union of all parsed critiques from the eight calls,
shuffled with a fixed seed and stripped of lens, call and severity. The judge labeled
every critique either with the planted flaw whose detection criterion it satisfied or as
unmatched; unmatched critiques were further labeled substantive (a specific, plausible
concern not in the ground truth), generic (boilerplate) or incorrect (factually wrong, or
misreading the analysis). Two judge models were used (gpt-5.6-sol and gpt-5.5, both
at high effort); both are also evaluated as reviewers, at medium effort. Agreement is
reported as Cohen’s κ [65]; the first judge is primary, and all headline results were
recomputed with the second judge and with the labels on which both judges agreed.

Outcomes and statistics. The primary outcome is flaw recall: the ~~loop advances~~
~~every iteration by default. When stricter behaviour is wanted, an optional~~
~~advancement gate~~ is the deadlock policy: while the merged result contains an

~~unresolved Critical critique~~ fraction of an item’s planted flaws identified by at least one critique of the condition. Secondary outcomes are recall by family and type, severity-aware recall (the flaw is identified by a merged critique of high or critical severity), the flaws found by only one panel lens, discrimination between flawed and clean items, measured by the AUROC of the merged grade and of a severity score that sums 8, ~~the run is not allowed to converge or complete and instead keeps revising;~~ bounded by the iteration ceiling so a genuinely irreconcilable panel terminates rather than looping forever. Ties are broken deterministically as above, so a split panel produces a stable, reproducible ordering rather than an undefined one.

~~Grade computation~~

An overall grade is computed from the merged critique list: ≥ 2 Critical $\Rightarrow$ F; ≥ 1 Critical $\Rightarrow$ D; ≥ 3 High $\Rightarrow$ D; ≥ 1 High $\Rightarrow$ C; ≥ 1 Medium $\Rightarrow$ B; otherwise A4, 2, 1 and 0 points for critical to info critiques, high and critical critiques on clean items, the composition of unmatched critiques, and tokens. Five contrasts were planned (C5–C1, C5–C3, C5–C4, C4–C2 and C3–C1) and tested with a paired bootstrap over items (10,000 resamples, repeats kept within item), with Holm correction [66] within each configuration and for the pooled analysis. The analysis plan was fixed before any reviewer call (S1 File); raw reviewer outputs, judge labels, analysis scripts and processed results are in S2 File.

~~Structured feedback and output cleaning~~ Merge evaluation

~~Artifact-aware feedback~~

A key finding from initial deployments was that generic feedback (“Previous iteration output: ... Continue with : task”) produced shallow revisions. We introduced *artifact-aware feedback* with two innovations. First, **reference inference**: when parsing reviewer critiques, the system infers artifact references (e.g., PROTOCOL.md) and section references (e.g., “Section 2.1 Controls”) from the critique text using pattern matching against known artifact names and section header patterns. These references are attached to each ReviewCritique as `artifact_ref` and `section_ref` fields. Second, **revision framing**: instead of the generic “Continue with: task” prompt,

~~revision iterations use structured framing:~~ Within each panel instance, two critiques from different lenses that the primary judge matched to the same planted flaw form a duplicate pair, and two critiques matched to different flaws form a non-duplicate pair; unmatched critiques are excluded. Each merging method was scored by pairwise precision, recall and F1 score (their harmonic mean), with its threshold selected by leave-one-scenario-out cross-validation (maximizing F1 on the other scenarios). The method with the best held-out F1 was pre-specified as the platform default. We also measured escalation validity: the proportion of multi-lens groups that correspond to a planted flaw. Adjudications, analysis code and results are in S2 File.

Stopping calibration

~~Output cleaning pipeline~~

~~LLM outputs frequently contain “thinking traces”—narration about the model’s intended actions (e.g., “I will read the file...”, “Let me check the data...”)~~ that contaminate research artifacts. ~~The output cleaning module removes these traces using regex-based line filtering:~~

- ~~Forward-looking narration: “I will...”, “I’ll...”, “Let me...”, “I need to...”, “I’m going to...”.~~
- ~~Tool-call narration: “I will read/search/check/verify...the...”.~~
- ~~Sequencing prefixes: “First, I will...”, “Next, I’ll...”.~~
- ~~Meta-commentary: “The following is...”.~~

~~The cleaner preserves code blocks (tracking “`” fences)`~~ To calibrate the stopping rule, we ran real executor-panel loops on the twelve flawed items for a fixed horizon without early stopping: the panel reviewed the original write-up, the executor revised it, and so on for five revisions, in two configurations (gpt-5.6-sol at medium effort and gpt-5.6-luna at low effort, for both executor and panel). Panels merged critiques with lexical Jaccard similarity at $\tau = 0.5$, the same setting as the benchmark’s severity outcomes; at this threshold few cross-lens duplicates are merged (Results), so the executor received the lenses’ critiques largely unmerged and few agreements were

escalated. Because no data were available, the executor could correct or withdraw claims, add caveats and specify analyses required before a claim could stand, but was instructed not to report new numerical results. After every revision a single judge (gpt-5.6-sol at high effort; the same model as the executor and panel of the first configuration) labeled each planted flaw as resolved, unresolved, or resolved by fabrication (the revision asserts results or analyses that were not performed) and listed newly introduced errors. Each candidate stopping rule was then replayed on the recorded trajectories with the platform’s own replay function, which applies the rule exactly as the engine does, giving its stopping point, whether it stopped while a planted flaw was unresolved (a premature stop) and the number of revisions used. The candidates were each of the three signals alone, any of them, all of them, and the absence of critical or high critiques combined with either grade stability or output similarity (with and without the advancement gate), all with the default window and minimum number of iterations; variants with a window of two and with similarity thresholds of 0.8 and 0.95; and fixed budgets of one to five revisions. The pre-specified selection rule chooses the rule with the lowest premature-stop rate among rules that stop before the horizon in at least half of the runs, breaking ties by fewer revisions. The analysis plan does not state whether the rule is applied within each configuration or to all runs together; because the platform has a single default, we applied it to the 24 runs pooled over both configurations and also report the selection within each configuration. The criterion counts only planted flaws; errors introduced by the revisions are reported for every rule but do not enter the selection. Loop trajectories, state judgments and replay code are in S2 File.

Executed case study

Question and data. We asked whether Geneformer attention encodes transcription-factor (TF)→target regulation beyond co-expression, expression-rank proximity and hub (degree) structure. One thousand cells were sampled from the immune compartment of Tabula Sapiens [67] (10x 3’ v3 chemistry only, stratified by cell type, fixed seed) and tokenized from raw unique molecular identifier (UMI) counts with the Geneformer V2 rank-value convention (normalization by total counts and by

each gene’s non-zero median in the pretraining corpus, descending rank, at most 4,markdown headers,096 tokens). Geneformer V2-104M (12 layers, 12 heads; Hugging Face snapshot fcd26c45) [2] was run in evaluation mode, and for a set of 1,322 genes (the 1,200 most frequently tokenized genes plus the TRRUST TFs present in at least 30% of cells) the attention from each query gene to each key gene was averaged over the cells in which both genes were present, per layer and per head. The resulting data kit also contains Pearson and Spearman co-expression, mean rank distances, co-presence counts and the TRRUST [68] and DoRothEA (confidence A–C) [69] edges among these genes (424 and substantive past-tense narration (“I have completed the analysis...”)). Excessive blank lines left by removals are collapsed. Cleaning is applied immediately after adapter output, before any downstream consumer (artifact parser,feedback formatter, consensus merger). 2,118 edges, excluding self-loops). Construction scripts, file checksums, package versions and sanity checks are in S3 File.

Severity stratification and lens weighting

Formatted feedback is stratified by severity—Critical and High critiques become *required fixes* the executor must address, while Medium and below become *suggested improvements*—so revision effort concentrates on what matters. Feedback is uniform across lenses by default; when a deployment trusts one lens more than another (for example, weighting the biological-plausibility lens for a tissue-specific study), an optional per-role weight biases the ranking so that lens’s critiques appear earlier in the required-fixes list without changing which critiques are reported. **Agent runs.** The `reviewer_consensus` workflow was run on the kit with verified execution (`sandbox_exec` backend; kit readable, network denied, 900-s wall-clock, 1,800-s CPU and 64-MiB file-size limits) for five executor iterations (an initial analysis and four revisions, the default revision budget), each followed by a panel review, with gpt-5.6-sol (two replicates) and gpt-5.5, all at medium effort and with LLM-adjudicated merging. A control run with gpt-5.6-sol and execution disabled measured what an executor reports without executing code. The task text described every kit file but contained no results. Convergence was disabled, so every run used this full horizon. Executor iterations are numbered 1–5 within each run, and each is followed by its panel review. For each executor iteration we extracted every reported

numerical result, checked whether it appears, at the reported precision, in the outputs of that or an earlier execution, and recorded whether it also occurs as a literal in the executed script.

Verified execution

By default an executor produces an analysis *plan*; a plan that is never run cannot be checked. When `code_execution_enabled` is set, MI-Workbench extracts the fenced Python blocks from an executor artifact and runs each in an isolated subprocess—a separate `python -I` interpreter in a fresh temporary directory, **Independent analysis.** Separately from the platform’s agents, we analyzed the same kit with our own code, written with LLM assistance (see Use of language models outside the evaluated system; S3 File). We measured attention symmetry per layer and head (correlation of A_{ij} with A_{ji} and relative asymmetry) and directionality on reference edges (paired comparison of TF-as-query with target-as-query attention). We measured recovery of TRRUST edges among all ordered TF–gene pairs by AUROC and by the area under the precision–recall curve (AUPRC), with TF-level bootstrap intervals, for attention and for co-expression, expression and degree baselines, also after residualizing attention on co-expression, expression and rank distance, and against a degree-preserving permutation null (1,000 rewirings that preserve every TF’s out-degree and every target’s in-degree). Multiple tests were corrected with the Benjamini–Hochberg procedure [70] or with max-statistic permutation tests. Two decompositions (separating gene-level from pair-specific attention, and comparing attention with co-expression on the same rewired edge sets) were added after the main results had been inspected and are reported as post hoc.

Evaluation of the agent runs. The runs were evaluated from their saved artifacts (prompts, executor scripts, execution outputs and panel critiques) after all runs had finished; no agent code was re-run. Numbers were extracted from the artifacts by script, and the assessment was drafted with LLM assistance and checked by a second agent that re-opened every cited source file and recomputed all counts (S3 File). A panel critique counted as a confirmed defect when the error it described could be confirmed in the cited code lines or outputs, and a wall-clock timeout, captured

~~stdout/stderr, and an output-size cap—then appends a concise execution report to the~~
~~feedback stream so the reviewer panel critiques executed results rather than an~~
~~untested plan. Execution is opt-in and deliberately bounded: it verifies deterministic,~~
~~self-contained snippets and is not a general sandbox for adversarial code, which would~~
~~require an operating-system-level jail around the process. Even so, it closes the loop~~
~~from proposed analysis to observed result and to critique of that result~~defect counted
as fixed when the code or outputs of a later execution no longer showed it; corrected
code was not re-run independently. Execution failures (crashes and timeouts), which
the execution report already showed to the reviewers, were distinguished from defects
that only a reading of the code or outputs could reveal. The 305 merged critiques of
the 15 executed reviews were not audited exhaustively, so the counts of confirmed
defects and of factually wrong critiques are lower bounds, and the panel’s precision is
not estimated. Each run’s final conclusion (that of its last successful iteration where
the final execution failed) was compared with the independent analysis on seven
claims: attention symmetry, TF→target directionality, recovery of TRRUST edges
above chance, recovery above co-expression, robustness of that advantage to degree
structure, a pair-specific signal after full adjustment, and causal interpretation. Each
claim was marked as agreeing, disagreeing, not addressed, or not directly comparable
when the run reached a conclusion about a different estimand. Because the runs chose
their own estimands, candidate sets and multiplicity corrections, agreement means the
same direction of conclusion, not the same test. Attention symmetry, which was not
part of the question, was checked only for whether a run asserted it. Statements that
something did not occur (that no run asserted attention symmetry, or that no panel
identified a verdict written into the code) rest on regular-expression searches of all
executor outputs, generated reports and critiques, supplemented by reading, so a
paraphrased mention could have been missed.

~~Provider-agnostic adapter layer~~Orchestration overhead

~~MI-Workbench defines a `BaseAdapter` abstract interface with three methods:~~
~~`run(request) → result`, `smoke_test() → status`.~~Control-plane overhead was
measured through the production runner with a real SQLite database and artifact

writing, using the mock backend with zero and 1-s artificial latency and 1–20 concurrent runs (five repeats per level) on an internal and an external drive. Panels in these measurements merged critiques lexically, so the additional adjudication call of the default merge is not included. After every batch, a database integrity check compared iteration counts across database rows, artifact directories and run metadata. Real-backend concurrency was measured in one batch at each of 1, 2, 4 and `is_available() → bool`. Four implementations are provided (Table ??8 simultaneous three-lens panels on one benchmark item (gpt-5.6-luna, low effort).

Adapter implementations and their CLI invocation patterns:

Use of language models outside the evaluated system

The mock adapter deserves special mention: it produces realistic MI research outputs with role detection from prompt content, grade progression ($C \rightarrow B \rightarrow B+ \rightarrow A-$), Apart from the models evaluated through the platform (S1 Table), Claude Opus 5.5 (Anthropic; model identifier claude-opus-5-5), used interactively through Claude Code 2.1.284, assisted in writing the benchmark items and their ground truth, the code of the independent case-study analysis, and synthetic metrics (AUROC, correlation, validation splits) with diminishing variation across iterations. This enables rapid prototyping and comprehensive testing without LLM API costs. The adapter registry provides singleton access and availability discovery, enabling the API and CLI to dynamically list available providers and their status. the synthesis of the case-study runs; separate instances of the same model, working in fresh contexts, checked the benchmark items and the synthesis against their source files. These outputs were validated in different ways. The benchmark was frozen before any reviewer call, and the judges applied its detection criteria; the independent analysis was cross-checked against the agents' own computations of the same quantities (Results); and every number in the synthesis is linked to its source file (S3 File). The same model also assisted with software development of the platform and with drafting and editing this manuscript. Token usage of these sessions is not reported.

~~Knowledge extraction~~ Implementation and ~~claim~~ ~~management~~ availability

MI-Workbench includes a knowledge extraction pipeline that mines structured claims from research artifacts. **Claim detection:** claims are detected via pattern matching on phrases such as “we found that...”, “results show...”, “AUROC = X”, and “ $p < 0.05$ ”. Detected claims are classified as *empirical*, *methodological*, *interpretive*, or *comparative*. **Uncertainty estimation:** each claim receives an uncertainty score $u \in [0, 1]$ (baseline 0.5) adjusted by strong evidence ($p < 0.001$: $u \leftarrow u - 0.2$); confidence intervals present ($u \leftarrow u - 0.1$) is implemented in Python 3.11 with FastAPI, aiosqlite, Click and Pydantic [71]. The backend comprises about 19,800 lines of code; 1,067 automated tests cover orchestration, consensus review, feedback, adapters, execution backends and persistence, and 68 further tests cover the evaluation harnesses (S1 Text). The source code, prompts, workflow presets, evaluation harnesses and figure scripts are available under the MIT license at <https://github.com/Biodyn-AI/mi-workbench>, together with instructions to reproduce every experiment (<docs/REPRODUCING.md>); the version described here is archived on Zenodo (<https://doi.org/10.5281/zenodo.23097814>) [72].

Results

What automated reviewers catch in planted-flaw analyses

We first asked how many planted flaws a reviewer configuration identifies, and whether a panel of three distinct lenses identifies more than a single lens sampled three times. All 1,296 reviewer calls ($18 \text{ items} \times 8 \text{ calls} \times 3 \text{ repeats} \times 3 \text{ model configurations}$) returned a parseable critique block (one call needed a retry), with a mean of 17.3 critiques per call. The two judges agreed closely: Cohen’s κ was 0.93 (95% CI 0.90–0.95) for the critique-level labels of 22, ~~hedging language such as~~ “suggests” or “may” ($u \leftarrow u + 0.1$), and large sample $n \geq 1000$ ($u \leftarrow u - 0.1$). 398 critiques and 0.84 (0.78–0.90) for whether each planted flaw was detected by each condition. Table 4 summarizes the main outcomes.

~~Falsification test generation: for each claim type, the system~~

Table 4. Benchmark outcomes by condition, pooled over the three model configurations.

Condition	Calls	Flaw (95% CI)	recall	Severity- aware recall	High or critical per clean item	Output tokens per item
C1 one rigor reviewer	1	0.81 (0.72–0.88)	0.74	6.0	2,469	
C2 one combined re- viewer	1	0.86 (0.76–0.94)	0.83	6.4	3,593	
C3 rigor ×3	3	0.90 (0.83–0.96)	0.86	19.4	7,416	
C4 combined ×3	3	0.91 (0.84–0.97)	0.90	19.4	10,889	
C5 three-lens panel	3	0.91 (0.86–0.97)	0.88	11.4	6,143	

Primary judge; 108 flawed and 54 clean item evaluations per condition. Severity outcomes use the pre-specified lexical merge ($\tau = 0.5$), which merges few duplicates, so they approximate unmerged counts (see Evaluation protocol). With TF-IDF merging at $\tau = 0.075$, high or critical critiques per clean item were 18.2 (C3), 21.8 (C4) and 11.8 (C5) (S2 Table).

~~auto-generates falsification suggestions—AUROC claims prompt stronger~~
~~baselines (mutual information, variance-based features), layer-specific~~
~~claims prompt split-sample validation, attention claims prompt~~
~~residualization tests,~~ Several calls catch more than one; distinct lenses do
not add recall at a matched number of calls. A single rigor reviewer identified
81% of planted flaws (pooled over configurations; 95% CI 72–88%), and a single
combined-checklist reviewer 86% (76–94%) (Fig 2A). Every three-call condition
identified more: 90% (83–96%) for three rigor samples, 91% (84–97%) for three
combined-checklist samples and 91% (86–97%) for the three-lens panel. Three samples
of the same reviewer exceeded one sample by 9.5 percentage points for the rigor lens
(95% CI 5.3–13.7) and by 5.3 points for the combined-checklist reviewer (2.1–9.3; both
Holm-adjusted $p = 0.001$); because each three-call condition contains the single call it
is compared with, these contrasts measure what the additional calls add (Materials
and methods). The panel exceeded a single rigor reviewer by 10.9 points (6.3–15.7;
 $p_{\text{Holm}} = 0.001$), but did not exceed the self-ensembles with the same number of calls:
the differences from three rigor samples and three combined samples were 1.4 (–1.9 to
4.4; $p_{\text{Holm}} = 0.82$) and ~~statistical claims prompt stringent multiple-testing correction.~~
~~Claim graph:~~ claims are stored in a relational database with support for inter-claim
relationships (supports, contradicts, extends) 0.2 points (–1.6 to 2.5; $p_{\text{Holm}} = 0.98$).
The result was robust to the judge: under the second judge, the panel exceeded a

single rigor reviewer by 8.8 points and differed from the two self-ensembles by +0.5 and evidence linking; a graph API enables visualization of the claim network; supporting identification of weakly-supported claims and contradictions. -0.5 points. In no configuration did the panel significantly exceed either self-ensemble. Its advantage over three rigor samples was largest for the low-effort configuration (+6.3 points, $p_{\text{Holm}} = 0.11$), whereas for gpt-5.5 three rigor samples exceeded the panel under both judges (2.8 points, $p_{\text{Holm}} = 0.14$, and 3.5 points, $p_{\text{Holm}} = 0.03$). For some models, therefore, spending two of the three calls on other lenses rather than on further rigor samples can cost a little recall. Overall, the gain of the panel over one reviewer is a gain from making three calls, not from making them through different lenses. This agrees with reports that budget-matched sampling and aggregation match multi-agent debate on reasoning tasks [23, 55, 56] and suggests the same holds for critique.

Reproducibility infrastructure

Fig 2. Planted-flaw benchmark. (A) Flaw recall by condition for each model configuration and pooled (points: means over items and repeats; bars: 95% item-bootstrap confidence intervals). (B) Pooled recall by flaw family and condition; numbers in parentheses are planted flaws per family. (C) High and critical critiques per clean item under the pre-specified lexical merge ($\tau = 0.5$), which recognizes few duplicates (bars: pooled means; points: configurations); every clean item drew at least one. (D) Pooled recall against mean output tokens per item. Conditions: one rigor reviewer, one combined-checklist reviewer, three rigor samples, three combined-checklist samples and the three-lens panel. Primary judge.

Artifact versioning

Each iteration writes artifacts to a structured directory. This provides full provenance: every revision of every artifact is preserved with its iteration context. The lenses largely overlap. Of the 432 planted flaw instances reviewed by the panel, 359 (83%) were identified by at least two of the three lenses, 10 (2.3%) by the rigor lens alone, 22 (5.1%) by the adversarial lens alone, 4 (0.9%) by the biological-plausibility lens alone and 37 (8.6%) by none; of the flaws the panel detected, 91% were raised by two or more lenses. Part of this overlap follows from the prompts, which share some items by design (Materials and methods); the remainder suggests, but does not establish, that the models we tested apply much of a general rigor checklist even when prompted for

adversarial or biological critique. Our design cannot separate these two sources, but either way lens specialization changed the emphasis of the critiques more than their coverage.

Git integration

Four git modes are supported: `none` (no version control), `commit_per_run` (single commit after run completion), `commit_per_iteration` (commit after each iteration), and `branch_per_run` (create a dedicated branch `run/{run_id}`).

Reproducibility packages

The `repropack` generator creates self-contained ZIP archives containing all artifacts from **Technical errors about the model itself are the blind spot.** The panel's recall was high for statistical (98%), confounding (97%), causal-logic (94%) and biological flaws (100%), and every three-call condition reached 93–100% in each of these families; single reviewers were less dependable there (82–97% per family), with the lowest recall for circular validation (64–67%) and, for the single rigor reviewer, effect-size neglect (61%). Recall was low for technical flaws in every condition (61% for the panel; Fig 2B). The deficit is specific: false statements about attention mechanics were caught almost always—the three planted instances of “dot-product attention is symmetric, so $A_{ij} = A_{ji}$ ” were identified by 96% of single rigor reviewers and by every multi-call condition—whereas false statements about the specific model or its tokenization (for example, that Geneformer uses binned expression values, or an incorrect layer count for a named model version) had a recall of only 17–28% across conditions (four planted instances; 95% CIs from 0% to 33–56%), with no consistent gain from additional calls or lenses. All detections were of the two statements about Geneformer; the two statements about scGPT (that it orders genes by expression rank and a wrong layer, head and dimension count for its whole-human checkpoint) were never identified in any condition. Together with the `run` (organized by iteration) near-universal detection of the attention-symmetry error, which no lens checklist names, this pattern suggests that reviewers check the logic of an analysis against general knowledge but do not reliably check its premises about the particular model being analyzed. This is an inference from four flaws, and two contributing factors

cannot be separated from it: the reviewers ran without tools and so could not look up model facts, and the rigor checklist mentions Geneformer’s rank-value encoding but no fact about scGPT. Whatever the cause, such premises are better verified by checks that do not rely on the reviewer’s recall, for example by checking architecture and tokenization claims against the model configuration in executed code.

The panel is less noisy. Reviewers raised far more concerns than there were planted flaws: across conditions only 19–29% of critiques of flawed items matched a planted flaw, 56–64% were specific but unplanted concerns, 11–22% were generic and 1–2% were judged incorrect. The panel had the highest share of critiques matching planted flaws (29% vs 19–21% for the other conditions) and the lowest share of generic critiques (11% vs 15–22%). On clean items, which contain no planted flaw, the panel raised a mean of 11.4 high or critical critiques per item, against 19.4 for both self-ensembles and 6.0–6.4 for single reviewers (Fig 2C); the difference held with TF-IDF merging at $\tau = 0.075$ (11.8 vs 18.2 and 21.8; ~~run_meta.json with run configuration~~ S2 Table). The panel also produced fewer output tokens than either self-ensemble (6,100 vs 7,400 and 10, ~~timing, 900~~ per item; Fig 2D) and took less summed call time (141 vs 170 and ~~cost; prompts_used.json with resolved-prompt templates; environment.json with Python version, installed packages, and OS details; git_state.json with branch, commit hash, and dirty status; and auto-generated dataset_card.md, reproducibility_checklist.md, and README.md.~~ 227 s per item). Input tokens, dominated by the three copies of the write-up under review, were similar (33,100 vs 32,700 and 37,900 per item), so total tokens were only 2% lower than for three rigor samples and 20% lower than for three combined-checklist samples. At equal recall, distinct lenses thus buy a more focused critique list with fewer output tokens and shorter calls; against a single-lens self-ensemble the total-token saving is small.

Implementation

MI-Workbench is implemented in Python 3.11 using FastAPI for the REST API, ~~aiosqlite for async database operations, Click for the CLI,~~ **The letter grade does not certify a clean analysis.** Severity scores separated flawed from clean items (AUROC 0.84 for the panel, 95% CI 0.79–0.90, and ~~Pydantic for data validation~~

(Table ?? 0.74–0.85 for the other conditions; the flawed version scored higher than its clean counterpart in 98% of matched comparisons). The panel’s grade had the highest AUROC (0.89, 0.82–0.95, vs 0.74–0.82, with overlapping intervals), but this advantage depended on the pre-specified lexical merge, which left the panel’s critiques almost entirely unmerged while merging and escalating some near-identical critiques within the self-ensembles: with TF-IDF merging at $\tau = 0.075$, the grade of every multi-call condition had an AUROC of 0.50 (next section). Yet clean items almost never received a passing grade: 268 of 270 clean-item grades across conditions were D or F, and the remaining two were C. Because every reviewer finds some high-severity concern in any analysis, the grade is at most informative as a ranking, not as an absolute criterion of acceptability, which has direct consequences for stopping rules (below).

Codebase statistics.

Merging paraphrased critiques

~~Key implementation decisions include: *async throughout*.~~ The panel’s value depends on recognizing when two lenses raise the same problem in different words, both to avoid duplicate required fixes and to escalate issues on which lenses agree. Within the 162 panel instances, the primary judge’s labels defined 1,342 cross-lens pairs of critiques identifying the same planted flaw and 3,214 pairs identifying different flaws. With word-level Jaccard similarity at a threshold set a priori without calibration ($\tau = 0.5$), merging found 1.2% of the duplicate pairs (pairwise F1 0.02; Fig 3B): paraphrased critiques share too few exact words to reach that threshold. TF-IDF similarity at its a priori threshold ($\tau = 0.3$) found 62% (F1 0.76), because down-weighting common words lets paraphrases that share a few informative terms pass. The failure lies mostly in the threshold rather than in lexical similarity as such. With thresholds selected by leave-one-scenario-out cross-validation (0.075–0.1 in every fold; 0.075 in ~~which~~ 11 of 12 folds for each method), lexical Jaccard reached a held-out F1 of 0.80 and TF-IDF cosine similarity 0.81, and LLM adjudication, which needs no threshold, reached 0.82 (Fig 3A). On these flaw-matched pairs all three methods almost never merged critiques of different planted flaws (precision 0.997–1.000) and differed mainly in recall (0.67–0.70). This precision covers only the 29% of panel critiques on flawed items that

matched a planted flaw; it does not show that the remaining 71%, unplanted concerns that carry no pair labels, were kept apart. The methods did differ in how much they merged overall: on flawed items the lexical methods formed 1,248 (Jaccard) and 1,259 (TF-IDF) multi-lens groups, against 858 for LLM adjudication, which left more critiques unmerged. Recall cannot exceed 0.74 in this design, because critiques from the same reviewer call are not merged under the platform default: when one lens raises a flaw twice and another lens raises it once, the single critique can join only one of the two, so one of the two cross-lens duplicate pairs cannot be recovered. In total, 349 of the 1,342 duplicate pairs are unreachable by construction; relative to that bound the methods recovered 91–95% of the mergeable pairs. LLM adjudication returned a valid partition for all I/O operations (database, adapter calls, file writes) use `async/await` for concurrency; *SQLite with WAL mode*, where write-ahead logging enables concurrent reads during long-running writes, validated under stress testing with 10 concurrent runs; an *event-driven architecture*, in which the engine emits events (`iteration_started`, `iteration_completed`, `budget_exceeded`, etc.) that are broadcast to WebSocket clients for real-time monitoring; and *crash recovery*, in which interrupted runs are detected on server restart and transitioned to resumable states¹⁶² panels at a cost of about 15,300 input and 1,300 output tokens per consensus step. The differences in F1 (0.01–0.02) are small, so we regard the three methods as comparable in duplicate detection; under the pre-specified selection rule (best held-out F1), LLM adjudication, the most conservative of the three, is the platform’s default merge, with TF-IDF at $\tau = 0.1$ as the zero-cost alternative. The optional sentence-embedding method, which requires a locally installed embedding model, was not included in this evaluation, and its default threshold is uncalibrated.

Results

We validated MI-Workbench on a substantive MI question: *do attention heads in Geneformer encode causal gene regulatory relationships, or do they merely reflect co-expression patterns?* **Agreement is informative.** Merged groups raised by two or more lenses contained a planted flaw far more often than single-lens groups (49% vs 5% with LLM adjudication; 40% vs 3% with calibrated Jaccard), and 98% of the

Fig 3. Merging duplicate critiques. (A) Pairwise precision, recall and F1 of duplicate detection, held out by leave-one-scenario-out cross-validation, for lexical Jaccard and TF-IDF similarity at cross-validated thresholds and for LLM adjudication. (B) Precision and recall against the similarity threshold over all panels (in sample). Open circles: uncalibrated thresholds set a priori (Jaccard 0.5, TF-IDF 0.3); filled circles: the threshold selected most often by cross-validation (0.075, in 11 of 12 folds for each method); dotted line: the platform default (0.1, the upper end of the fold-selected range). Recall cannot exceed 0.74 because critiques from the same reviewer call are not merged (see Results).

planted flaws raised by two or more lenses were escalated. Escalation on agreement therefore concentrates severity on real problems (the escalation threshold itself, two distinct lenses, was not varied). It also has a side effect that the benchmark makes visible: when the benchmark grades were recomputed with TF-IDF merging at $\tau = 0.075$ and escalation on agreement, every multi-call condition assigned grade F to every item, clean ones included, so the grade did not separate flawed from clean items (AUROC 0.50, against 0.66–0.76 for the underlying severity score; S2 Table). The grade discrimination reported above (AUROC 0.74–0.89 for the three-call conditions) therefore holds only when duplicates are largely left unmerged. TF-IDF stands in for LLM adjudication in this analysis because the adjudicator was run only on panel instances; the two methods had similar merge accuracy (F1 0.81 vs 0.82) and both escalated 98% of the flaws raised by two or more lenses. Over-merging may contribute to the saturation: on clean items, TF-IDF merging formed more multi-lens groups than LLM adjudication (524 vs 308 over the 54 clean panels), and the ground truth cannot show whether such groups join one concern or several. We return to the consequences for grading below.

Experimental setup

~~We used the Gemini CLI adapter to run end-to-end experiments across three loop presets (Table ??).~~

When a review loop should stop

~~Case study runs. All runs used the Gemini CLI adapter with OAuth authentication.~~
The third question was when a revision loop should stop, given that reviewers never run out of concerns. We recorded 24 real loops (12 flawed items $\times$ 2 configurations),

each with five revisions and six panel reviews, and judged the state of every planted flaw after every revision (Fig 4).

Key findings from automated analysis

The platform autonomously identified three primary confounders explaining the attention-co-expression correlation. **(1) MLM objective confounder:** the executor agent correctly identified that Geneformer’s masked language modeling objective incentivizes attending to co-expression cliques rather than causal regulators, because co-expressed genes are the most statistically powerful predictors of masked token identity. **(2) Rank-value encoding artifacts:** the system identified that Geneformer’s rank-value tokenization creates an inductive bias where positional proximity in the sequence corresponds to expression magnitude similarity, producing false-positive attention between co-abundant genes (homophily) rather than regulatory links. **(3) Attention symmetry:** standard dot-product attention softmax($QK^\top$) is inherently bidirectional unless the model learns distinct query/key projections; the executor noted that $A_{ij} \approx A_{ji}$ fails to capture the directional nature of gene regulation (TF $\rightarrow$ Target).

Fig 4. Stopping calibration on 24 real revision loops. (A) Planted flaws left unresolved per item after each revision, as judged. (B) Newly introduced errors per item. (C) Critical and high critiques per panel review (lexical merging, $\tau = 0.5$). (D) Premature-stop rate against mean revisions used for each candidate stopping rule, pooled over both configurations; the selected default (a budget of four revisions) is filled. Grade stability alone and the rule that stops when any of the three signals holds share one point (diamond), because no other signal fired. Rules that require the absence of high-severity critiques or a stable executor output never stopped within five revisions.

Planted flaws were fixed early; new errors accumulated later. The first revision resolved most planted flaws: all four were resolved in 83% of items with gpt-5.6-sol and 67% with gpt-5.6-luna, leaving 0.17 and 0.33 unresolved flaws per item. Later revisions added little (0.08 unresolved flaws per item after four revisions in both configurations), and no planted flaw was resolved by asserting an analysis that had not been done. Every flaw that remained unresolved after a revision was a false statement about the model or its tokenization, the flaw type that reviewers also rarely detected in the benchmark; the panel had not flagged these statements, which most

likely explains why the executor left them unchanged. Meanwhile the judge found newly introduced errors, and these accumulated: from 0.17 and 0.67 per item after the first revision to 1.75 and 1.83 after the fourth, the platform’s default budget, and 1.00 and 1.92 after the fifth (Fig 4B). With 12 items per configuration we do not read the fall for gpt-5.6-sol at the fifth revision as a reversal; in both configurations the write-ups still contained far more new errors than after the first revision. The two configurations erred differently. Statements that presented an analysis, control or result as done although the original write-up did not contain it (labeled fabrications by the judge) made up about half of the new errors with gpt-5.6-sol from the third revision onwards (0.75 of 1.25 per item after the third revision and 0.92 of 1.75 after the fourth), although the executor had been instructed not to report new results; with gpt-5.6-luna they stayed at or below 0.17 per item. Most of these statements misdescribed what the write-up had done, for example by crediting it with procedures or thresholds it never contained, rather than reporting new numerical results. Write-ups grew from about 6,200 characters to 25,000–56,000 after the fourth revision, mostly through added caveats and plans for pending analyses.

~~Iterative refinement quality~~

~~The structured feedback mechanism produced measurable quality improvements across iterations. In iteration 1, the executor produced an initial analysis with three confounders and a protocol outline with null hypotheses. In iteration 2, the reviewer identified missing controls—a cell-type composition confounder, an unspecified dataset, The panel’s signals separate the original from the revised write-ups but do not track later changes.~~ The number of critical and high critiques fell by about half after the first revision (from 23.8 to 11.8 per review with gpt-5.6-sol and from 16.3 to 8.8 with gpt-5.6-luna). Pooled over all six reviews of each loop, it therefore correlated with the number of unresolved planted flaws (Spearman ρ 0.54 with gpt-5.6-sol and 0.49 with gpt-5.6-luna; 0.66 and ~~missing BH-FDR correction details.~~ In iteration 3, the executor revision added a within-cluster partial correlation control, specified the Genecorpus-30M human bone marrow subset, defined a mean rank baseline, and set a random seed. Critically, the revision iteration produced *targeted*

~~changes to the specific artifacts and sections referenced in the feedback, rather than~~
~~rewriting from scratch—a direct result of the artifact-aware feedback mechanism. 0.70,~~
respectively, for critical critiques alone). This correlation reflects the contrast between
the original write-up, in which all four flaws were unresolved, and the revisions; after
the first revision the count neither followed the few remaining planted flaws nor rose
as new errors accumulated, but leveled off at 7–12 per review and never reached zero
(Fig 4C). The merged grade was F for 23 of the 24 original write-ups, rose to D for all
24 first revisions and was D in 113 of the 120 reviews of revised write-ups (6 were F
and 1 was C). These grade levels depend on the merge: the loops’ panels merged at
 $\tau = 0.5$, which escalated few agreements, whereas with effective merging the
benchmark panels graded every item F (see Merging paraphrased critiques). Because
critical and high critiques never disappeared and consecutive executor outputs never
became similar enough, every candidate rule that requires the absence of high-severity
critiques or a stable executor output never stopped within five revisions, with or
without the advancement gate. The only adaptive signal that fired was grade stability,
which in practice detects that the grade has stopped changing: it stopped after a
mean of 3.0 revisions and stopped prematurely—while a planted flaw was still
unresolved—in 3 of 24 runs (Fig 4D). A fixed budget of three revisions performed
identically (3 of 24), and a fixed budget of four revisions stopped prematurely in 2 of
24 runs. Applied to the 24 runs together, the pre-specified selection rule therefore sets
the platform default to a budget of four revisions after the initial submission, followed
by a final review; the adaptive rule remains available. The budget rules use only the
revision count and therefore do not depend on the merge method. Within
configurations the choice is less clear-cut: with gpt-5.6-luna, four revisions (1 of 12
premature stops) did better than three revisions and grade stability (2 of 12 each),
whereas with gpt-5.6-sol both budgets and grade stability each stopped prematurely in
1 of 12 runs, and the tie-break alone would select three revisions. The difference
between the candidates rests on one run of 24 and comes entirely from flaws the panel
does not detect, so no stopping signal derived from the panel could have prevented
those premature stops. The criterion also does not count introduced errors, and by
that measure four revisions is the most costly eligible rule: 1.75 and 1.83 new errors
per item, against 1.25 and 1.58 after three revisions and 0.17 and 0.67 after one.

Output quality assessment

Post-hoc validation by domain experts confirmed that all three identified confounders are well-documented in **Interpretation**. With current LLM reviewers, stopping is a budget decision rather than a convergence decision: the panel’s assessment improves sharply after the first revision and then plateaus at a level that would never pass an absolute threshold, while additional revisions trade a small reduction in unresolved planted flaws for an accumulation of new errors that the panel’s critique counts did not register. The letter grade can show this trajectory only when merging is weak; with effective merging it saturates at F, so it supports neither use as an acceptance criterion nor use as a stopping signal. A loop’s output should accordingly be treated as a draft for human acceptance. In these loops, in which the executor could only revise text, revisions beyond the first removed fewer planted flaws (0.08–0.25 per item between the first revision and the fourth) than the new errors they introduced (1.2–1.6 per item). The default of four revisions therefore minimizes planted flaws left unresolved at the price of more introduced errors. When introduced errors weigh more, a budget of one revision is preferable: it left 0.17 and 0.33 unresolved planted flaws and introduced 0.17 and 0.67 new errors per item. Executed loops may behave differently, because a revision can rerun a corrected analysis rather than only reword a claim; in the executed case study, confirmed defects were still being fixed in the fourth revision (see Executed agent runs). For these reasons the budget is a run-level setting.

Case study: does Geneformer attention encode regulation?

The fourth question was whether the loop reaches sound, auditable conclusions on a real interpretability question when the executor runs its own code. We asked whether Geneformer attention encodes transcription-factor (TF)→target regulation beyond co-expression, expression-rank proximity and hub (degree) structure, using a data kit built from 1,000 Tabula Sapiens immune cells (Materials and methods; S3 File). Whether the agents answered this question correctly can be judged only against a known answer, so we first report our own analysis of the kit and then the agents’ executed runs.

Independent analysis

Our own analysis of the kit serves as the reference against which the agents' executed analyses are judged (Fig 5; S3 File).

Fig 5. Independent analysis of the case-study data. (A) Spearman correlation between A_{ij} and A_{ji} by layer for head-averaged attention (line) and for the 144 individual heads (dots); a value of 1 would indicate symmetric attention. (B) Recovery of TRRUST edges (AUROC with TF-bootstrap 95% confidence intervals) by symmetrized attention per layer, the degree-preserving null (mean and 95% interval) and the absolute Spearman co-expression baseline. (C) Post hoc: the pair-specific component of attention after residualization on co-expression, expression and rank distance, against its degree-preserving null.

Geneformer attention is not symmetric. Across 873,181 gene pairs co-present in at least 50 cells, the head-averaged attention from gene i to gene j was only weakly correlated with the attention from j to i in every layer (Spearman ρ 0.16–0.32; median relative asymmetry $|A_{ij} - A_{ji}|/(A_{ij} + A_{ji})$ 0.14–0.31), and for 11–46% of pairs one direction exceeded the other at least twofold (Fig 5A). Individual heads were less symmetric still (median ρ 0.10 across the 144 heads, range –0.22 to 0.75). This is what the architecture implies: queries and keys use different learned projections, and each query's attention is normalized by its own row-wise softmax [73, 74], so $A_{ij} = A_{ji}$ is neither guaranteed nor observed. On TRRUST edges the TF-as-query direction received more attention than the target-as-query direction in several later layers (layer 12 median $\log_2$ ratio 0.53, 95% CI 0.35–0.75), but TF–gene pairs that are not edges showed a similar bias in most layers, so the direction of attention mostly reflects which token is a TF, not which pairs are regulatory.

Apparent edge recovery is mostly hub structure. Among 310,435 ordered TF–gene candidate pairs (235 TRRUST TFs; 424 TRRUST edges, base rate 0.14%), symmetrized attention ranked edges above non-edges in every layer, best in layer 3 (AUROC 0.673, 95% CI 0.631–0.708; AUPRC 0.0037, 2.7 times the base rate), and above absolute co-expression (AUROC 0.545, 0.511–0.575) in layers 1–7 (Benjamini–Hochberg $q < 0.05$). Rank proximity explained little of attention (Spearman ρ with rank distance –0.29 to –0.05 across layers). The decisive control was the degree-preserving null, which rewires the reference edges while keeping every TF's number of targets and every target's number of regulators: it alone reached an

AUROC of 0.645 in layer 3 and reproduced 64–102% of attention’s AUROC above 0.5 across layers. Attention still exceeded this null by a small margin in six of the literature [2, 7]; that the proposed null hypotheses (H_0^1 12 layers (0.017–0.046 AUROC; Benjamini–Hochberg $q < 0.05$; layer 3: 0.673 vs 0.645, max-statistic $p = 0.004$ over layers). A target’s in-degree in TRRUST, computed without the scored edge, reached an AUROC of 0.829 by itself. In post hoc decompositions, the gene-level component of attention (how much a gene attends and is attended to overall) matched the null exactly (layer 3: 0.654 vs 0.653), and attention’s advantage over co-expression dominance, H_0^2 : attention symmetry, H_0^3 : degreebias) are scientifically appropriate; that statistical framework choices (AUPRC over AUROC for sparse networks; BH-FDR correction) reflect current best practices; and that the output contained no thinking traces (validated by comparison with pre-cleaning runs where traces contaminated 4/4 artifact files).—was no larger than the advantage expected from degree structure in any layer (0 of 36 layer-by-orientation tests). A pair-specific component remained against the degree null in layers 1–6 (layer 3: AUROC 0.592 vs null 0.551, $q = 0.006$), but after further adjustment for co-expression, expression and rank distance it was not detectable with TRRUST (0 of 36 tests; best 0.612 vs 0.575, $q = 0.07$) and was detectable with the larger DoRotheA A–C reference (24 of 36 tests), with an excess of at most about 0.04 AUROC over the null in either reference.

Orchestration evaluation

The case study exercises one task; to characterise the orchestrator independently of any single question, we run a set of controlled experiments. Because a full LLM run is stochastic and costly, these use a deterministic mock adapter that emits, per reviewer lens, a fixed critique set with **Interpretation**. The direct evidence, from the pre-specified tests, is that reference degree structure alone reproduces most of the agreement between Geneformer attention and curated regulatory edges, although attention retains a small, controlled overlap; this isolates the *mechanism* and its *scaling* reproducibly, and we then corroborate the panel with a live Gemini backend. The mock experiments make no claim about research quality—that rests on the case study and the live corroborationsignificant excess over the degree-preserving null. Our

inference, which rests on the post hoc decompositions, is that this agreement is driven mainly by which genes are hubs in the reference and which genes attend or are attended to strongly, rather than by the specific TF–target pairs, and that a small pair-specific component exists but is close to the detection limit of the available references; the difference between TRRUST and DoRothEA is consistent with the larger reference giving more power, although DoRothEA’s lower-confidence levels derive partly from expression-based inference, so residual non-linear co-expression could contribute. A plausible but untested explanation for the hub-level agreement is that broadly expressed, well-studied genes are both over-annotated in curated databases and preferentially attended. For the purpose of this study, these results define what a correct automated analysis of the kit should conclude: raw edge recovery overstates regulatory information, and any claim of TF–target encoding must survive a degree-preserving null. Although symmetry was not part of the question, no correct analysis can treat attention as symmetric.

~~Does a panel add over a single reviewer? We vary the panel size and measure how many distinct critiques survive merging~~

Executed agent runs

We then ran the full loop on this question with verified execution: three runs (gpt-5.6-sol, two replicates; gpt-5.5), each with five executor iterations (an initial analysis and four revisions) and five panel reviews, and a control run with execution disabled (Table ??). ~~A single rigour reviewer surfaces three issues; adding the adversarial lens raises this to five and , because both lenses independently flag the missing effect-size issue, agreement escalates it to Critical (grade F) ; adding the biological-plausibility lens raises coverage to seven. Coverage grows monotonically with panel size, and agreement upgrades severity exactly when independent lenses concur—evidence that a panel is not redundant with a single well-prompted reviewer.~~ 5; S1 Table, S3 File). Each run used about 0.94–0.98 million tokens over 25 agent calls and took 43–91 minutes.

~~Are the lenses complementary or redundant? Treating each lens’s critique set as a bag of issues,~~ Every reported number came from executed code. All 15 executor responses in the executed runs consisted of a single analysis script without

Reviewer-panel ablation (deterministic harness, fixed quality tier). Distinct critiques surviving the merge, the number raised by more than one lens, and the resulting grade, as the panel grows.

Table 5. Executed case-study runs.

Run	Model	Executions (failed)	Tokens (M)	Traced results	Final conclusion on the TRRUST question
1	gpt-5.6-sol	5 (1)	0.94	460/460	No information beyond the nuisance model; supported by adequately resampled nulls in iteration 4, written as a literal in iteration 5
2	gpt-5.6-sol	5 (1)	0.98	149/149	Not identifiable from aggregate attention (verdict written as a literal; 4 matched edges left)
3	gpt-5.5	5 (2)	0.96	3/3	No positive association beyond confounders; reference degree dominates (computed from results)
Control	gpt-5.6-sol	none (disabled)	0.39	–	Declined to report results in every iteration

Each run: five executor iterations (an initial analysis and four revisions), each followed by a panel review. Traced results: numerical results in the code-generated reports of all five iterations that appear in executed output, out of all such results. Run 3’s reports cited keys of its results file instead of stating values; its three traced numbers are the 95% confidence level of its intervals. For runs 2 and 3 the final execution failed, and the conclusion column reports the last successful iteration. Run identifiers in S1 Table and S3 File: sol_A (run 1), sol_B (run 2), gpt55 (run 3) and sol_planonly (control).

prose, so every number in the write-ups was printed by the code or written by it to

report files. Tracing each number in the code-generated reports to the execution

outputs found 612 results, all present in executed output (0 untraceable; S3 File).

Because the code wrote the reports, this agreement is largely built in; what the trace

can detect is a reported number that sits as a literal in the code instead of being

computed. Of the 612 results, 595 did not occur as literals; the other 17 were analysis

settings or labels that the code printed verbatim (for example the significance level

0.05, the 95% confidence level and numbers of resamples), and no analysis result was

hard-coded. Where an agent and our independent analysis computed the same

quantity, they agreed, which shows that the values were computed from the data: the

36 layer-level AUROCs of the gpt-5.5 run (run 3) matched ours to within 5×10^{-8} ,

and those of the first gpt-5.6-sol run (run 1) to within 0.0011 (its co-presence filter

differed slightly). Eleven of the 15 executions completed; four failed (two 900-s

timeouts, one 64-MiB file-size limit, one programming error), and each failure was

reported to the reviewers and, for the two failures before the final iteration, to the

next executor. In the control run without execution, the executor declined in all five

iterations to report results, stating that it could not compute them; it neither

1236

1237

1238

1239

1240

1241

1242

1243

1244

1245

1246

1247

1248

1249

1250

1251

1252

fabricated numbers nor produced an analysis plan.

The panel caught real errors in executed analyses. Across the three executed runs, the panel identified 29 defects in the code or its outputs that we confirmed against the cited code lines and outputs (Materials and methods). Four of them were execution failures that the execution report already showed and that the panel diagnosed and relayed with a fix; the other 25 were analytical defects that only a reading of the code or its outputs could reveal. Twenty of the 29 were fixed before the run ended, 19 of them by the next execution that completed (for the pairwise Jaccard overlap is 0.20 for rigour-adversarial and 0.00 for both pairings involving biological plausibility; the union of all three lenses (seven distinct issues) is nearly the sum of their individual counts (eight) first-iteration defects of two runs this was two iterations later, because the intervening execution had crashed or timed out) and one only after a second panel raised it again. Three were fixed incompletely (one partially, one in code that never ran, one fixed and later broken again), and six were not fixed, four of them because the final panel raised them. The same error appeared independently in all three runs at the first iteration: degree covariates computed from the TRRUST labels, including the edge being predicted—a circular control that inflated the baseline (degree-only AUROC 0.95 in one run). Other caught defects included a verdict rule that selected the smallest p -value regardless of the sign of the effect, a graph-rewiring routine that could never accept a swap, a checksum verification that silently checked nothing, pair-level tests that treated 166, so the lenses overlap little and the biology lens in particular contributes issues the others never raise. The roles are complementary by construction, not duplicative 446 dependent pairs as independent, and null distributions compared at mismatched numbers of resamples. The only positive verdict among the 15 executed iterations—an association “beyond matched confounders” in the first iteration of one run—was withdrawn after the panel showed that the remaining imbalance in target in-degree exceeded the attention effect.

Where the agents ended up. For TRRUST, two runs reached the same conclusion as our independent analysis by different routes: no attention signal remained once co-expression, rank distance and degree structure were accounted for. The gpt-5.5 run computed this verdict from its results (best positive effect $q = 0.059$, max-statistic $p = 0.87$, cross-validated gain in AUPRC -0.0004 , 95% CI -0.0023 to

0.0013) and stated that reference degree dominated the predictive signal. The first gpt-5.6-sol run supported the same conclusion with adequately resampled nulls in its fourth iteration ($p = 0.56\text{--}0.73$), but in its final iteration it cut the nulls to 15 and 10 resamples, which could no longer reject at 0.05, and wrote its verdict as a literal in the code. That verdict did not reflect its own final estimate: the gain in AUPRC was 0.0046, with a 95% interval that excluded zero (0.0017–0.0076, a t -interval over 15 overlapping splits), while the null replicates averaged 0.0022–0.0030. The final iteration alone was therefore inconclusive rather than negative. The second gpt-5.6-sol run (run 2) moved to increasingly strict matching that retained only 4 TRRUST edges and declared the question not identifiable from aggregate attention. No final write-up reported the positive part of the reference result—that raw attention ranks TRRUST edges above chance and above co-expression—although the numbers were present in the agents’ own tables, and none tested the small pair-specific signal that our analysis detects with DoRothEA. Attention symmetry was not part of the question; no run measured it, and none asserted that attention is symmetric.

Convergence Failure modes. Over 20 independent runs of the consensus loop, every run terminated by convergence rather than the iteration ceiling, at a median of 8 iterations. The loop did not converge. Panels of the executed runs returned 15–24 merged critiques per review and graded 13 of the 15 reviews F, including the final review of every run (the gpt-5.5 run received D at its third and fourth reviews; all five reviews of the control run were also F); a recurring top-ranked request was for data the kit does not contain (cell-type-specific attention, immune-specific regulatory labels). Executors responded by expanding their scripts (from 1, each time via the grade-stabilization signal. The loop reliably detects when further iteration is unproductive. 025 to 2,233 lines in one run, 489 to 1,359 in another), and three of the four failed executions occurred in iterations that added large resampling machinery in response to critiques. Two runs wrote their final verdict on the research question as a literal in the code, so the conclusion could not respond to the computed numbers, and no panel critique identified the literal. The panel also made at least three factually wrong critiques (for example, misattributing a DoRothEA edge to TRRUST). The number trace shows that reported numbers came from executed code; it does not show that conclusions follow from them.

~~Load and scalability.~~ Although each run is coordinated centrally, independent runs execute concurrently. Driving 1 to

Orchestration overhead and concurrency

Because every agent call goes through one coordinator, we measured what coordination itself costs (S1 Fig). Through the production runner, with a real database and artifact writing and an instantaneous mock backend, the control plane added a median of 30 ms per iteration for a single run on the internal drive, rising to 650 ms with 20 concurrent runs through the orchestrator (Fig 2), aggregate throughput rises from about 110 to 376 iterations/s while p95 per-run latency grows sub-proportionally (from 55 as database writes and artifact I/O queued; with a 1-s backend delay the corresponding values were 51 and 610 ms to 314 ms. Other jobs ran on the workstation throughout (1-min load average 160–360 on 10 cores), so these values probably overestimate the overhead on an idle machine. On the slower external drive the median overhead reached about 0.9 s per iteration and, with the instantaneous backend, did not rise monotonically with the number of runs (S1 Fig), which points to substantial measurement noise from the shared machine and drive. Against the latency of a real reviewer call in the benchmark (median 51 ms at 20× concurrency), and a database integrity check passes after every level. The centralized coordinator therefore scales across concurrent investigations, with tail latency degrading gracefully rather than collapsing—s, mean 60 s over 1,296 calls), coordination accounts for well under 1% of a single run’s time and about 1% at 20 concurrent runs. After every batch, an integrity check found the database rows, artifact directories and run metadata consistent for all runs (104 of 104 checks). With the real backend, 1, 2, 4 and 8 simultaneous three-lens panels, run as one batch per level, completed without failures, retries or rate-limit errors, and throughput rose from 4.6 to 30.6 calls per minute, 84% of linear scaling at eight panels; the four-panel batch was slower than the eight-panel batch (median call latency 53 against 39 s), so single batches also reflect provider-side latency variation. Within the range tested (up to 20 concurrent mock runs and eight concurrent real panels), run time was therefore dominated by provider latency rather than by the coordinator. No rate limit was

reached, so these measurements do not show at what concurrency the provider’s rate limits or the cost of each call become binding; that these, rather than the coordinator, set the practical limit of the centralized design is an inference, and one that holds only while the coordinator’s overhead, which grew with the number of concurrent runs, stays small relative to call latency.

~~Orchestrator load scaling. (a) Aggregate throughput and (b) p95 per-run latency as the number of concurrent runs increases from 1 to 20. Throughput rises with concurrency while tail latency grows sub-proportionally.~~

Discussion

~~Live-backend corroboration~~ **What a reviewer panel buys.** ~~To confirm the wired panel behaves as characterised with a real model, we ran~~ On a benchmark with known flaws, every three-call condition found more planted flaws than a single reviewer (90–91% against 81% for a single rigor reviewer and 86% for a single combined-checklist reviewer, that is, about ten and five percentage points more), but the three-lens panel ~~once over a fixed Geneformer analysis artifact using the Gemini backend. All three lenses returned; the rigour lens alone produced 12 critiques, found no more than the same lens—or a reviewer holding all three checklists—sampled three times. The recall gain of the panel therefore comes from making several calls, the same mechanism that underlies self-consistency and the full panel produced 23 distinct merged critiques—roughly double the coverage—consistent with the ablation. Notably, verbatim Jaccard deduplication rarely merged critiques across lenses on free-text output (the lenses phrase overlapping concerns differently), which both underlines the coverage benefit of a panel and motivates semanticsampling-and-voting ensembles [55, 56], and it agrees with evidence that multi-agent debate rarely beats budget-matched sampling [22, 23]. Distinct lenses did change the output in a useful way: at equal recall the panel produced fewer generic critiques, a larger share of critiques matching real flaws and about 40% fewer high-severity alarms on clean items than the self-ensembles (11.4 vs 19.4 per item), with either merge setting, and it used fewer output tokens. Its letter grade also separated flawed from clean items best, but only while duplicates remained essentially unmerged: its severity score separated them~~

no better than three combined-checklist samples (AUROC 0.84 vs 0.85), and once
duplicates were merged every multi-call grade saturated at F. The lenses overlapped
heavily (83% of planted flaw instances, and 91% of those the panel detected, were
found by at least two lenses), partly because their prompts share items and plausibly
also because the models we tested apply a general checklist whatever lens they are
given; the benchmark cannot separate the two, but in either case the case for a panel
rests mainly on focus, with a smaller saving in output tokens and call time, rather
than ~~lexical, deduplication as a refinement~~ on coverage. For users, this means that the
number of reviewer calls, and for a single call a fuller checklist, are the main levers for
recall, although extra calls add less when the single reviewer is already strong, and the
choice between a panel and a self-ensemble can be made on noise and output cost.

Discussion and conclusion

What reviewers miss. With three calls, reviewers reliably caught statistical,
confounding, causal and biological flaws (93–100% per family in every three-call
condition), including the false premise that dot-product attention is symmetric; single
reviewers were less dependable on some flaw types, missing a third or more of
circular-validation flaws and, with the rigor lens alone, of effect-size flaws. By contrast,
reviewers found only about one in five false statements about the specific model or its
tokenization, regardless of the number of calls. These are premises a reviewer can only
check if it knows, or looks up, the model’s architecture and training procedure, and
our reviewers ran without tools. The estimate rests on four planted statements, two of
which (both about scGPT) were never found, so it shows the weakness without
measuring its size precisely. Additional reviewer calls did not fix it. We expect
grounding to help, but did not test it: requiring the executor to read architecture and
tokenization facts from the model configuration in executed code would turn them
into checkable outputs rather than assertions, and giving reviewers access to model
documentation would let them look such facts up.

~~MI-Workbench shows that a centrally orchestrated team of LLM agents can carry
out mechanistic-interpretability investigations that are rigorous, reproducible, and
inspectable. The evaluation supports its core design choices: a reviewer panel surfaces~~

~~strictly more, and complementary, critiques than a single reviewer; agreement between
lenses escalates severity where it should; the loop reliably detects convergence ; and
the centralized coordinator scales across concurrent runs with graceful tail-latency
growth. The provider-agnostic design insulates workflows from any one vendor, and
the accompanying test suite exercises the orchestration, consensus, feedback, and
execution paths.~~

~~We deliberately trade agent autonomy for a deterministic, auditable control flow,
which is the right trade for automated scientific critique: a run’s sequence of agent
calls follows from its workflow graph, every artifact and prompt is persisted, and any
run can be repackaged for independent reproduction. The domain specificity lives in
swappable prompts and configurable reviewer lenses rather than in the engine, so the
same coordinator drives other domains and other panel compositions; the evaluation
experiments are themselves task-independent evidence of this separation.~~Merging,
agreement and grades. Whether paraphrased critiques are recognized as duplicates
depended mainly on calibration: word-overlap (Jaccard) similarity at an uncalibrated
threshold merged almost nothing, while the same method at a calibrated threshold
came within 0.02 F1 of LLM adjudication. Agreement between lenses was
informative—groups raised by several lenses were about ten times as likely to
correspond to a planted flaw—which supports escalating agreed issues. At the same
time, the benchmark exposed a property of LLM review that any grading scheme must
accommodate: reviewers found high-severity concerns in every clean analysis. A
count-based letter grade can rank analyses, but it cannot certify one as acceptable,
and once duplicates are merged and agreements escalated it saturates. We therefore
treat the grade as an ordinal summary for monitoring and do not use an absolute
grade threshold as a default stopping criterion.

~~Several limitations remain. Verified execution is opt-in and intentionally
bounded—it runs deterministic, self-contained snippets in an isolated subprocess but
is not a hardened sandbox for adversarial code.~~When to stop. The stopping
calibration showed why convergence rules built on reviewer signals fail with current
models: the panel’s assessment improved sharply after the first revision and then
stayed on a plateau that no absolute threshold would accept, and the only adaptive
signal that ever fired—grade stability—merely detected that plateau. The few planted

flaws that survived a revision were flaws the panel had not flagged, so no rule based on
 the panel’s output could have caught them, while revisions after the first introduced
 more new errors than the planted flaws they removed. The pre-specified rule, applied
 to the runs of both configurations together, selected a budget of four revisions. The
 margin is one run of 24, and with gpt-5.6-sol alone three revisions did as well;
 moreover, the rule counts only planted flaws left unresolved, so it favors completeness
 over minimal editing: a budget of one revision introduced the fewest new errors at the
 cost of slightly more unresolved planted flaws. We therefore set a fixed budget of four
 revisions as the default and regard the length of the budget as a choice that belongs to
 the user. The executed case study, which ~~would require an operating-system-level jail.~~
~~Critique deduplication is lexical (word-level Jaccard); as the live-backend~~
~~corroboration shows, real reviewers phrase-overlapping concerns differently, so~~
~~semantic deduplication via embeddings would merge and escalate agreement more~~
~~faithfully and would also sharpen the convergence signal. Cross-provider cost~~
~~accounting depends on adapter-level reporting and is not yet standardized. Finally,~~
~~coordination is centralized: this is a feature for reproducibility but caps single-run~~
~~parallelism at the reviewer panel~~ used the same budget with LLM-adjudicated merging,
 did not converge either: its panels graded 13 of the 15 reviews of the executed runs F,
 including the final review of every run. An advancement gate on unresolved critical
 critiques is only useful once severity is calibrated, which our results suggest it is not.

What the case study shows. Verified execution did what it was designed to do.
 Every number the executed runs reported came from executed code, and the few that
 also appear as literals in the scripts are analysis settings; the agents’ computations
 agreed with an independent analysis of the same data, and the panel found and the
 executors fixed real analytical errors, including a circular degree control that all three
 runs introduced independently. In the one control run without execution, the executor
 declined to report results rather than inventing them. Because that run produced
 neither numbers nor a plan, the case study does not estimate how much verified
 execution reduces unsupported claims; it shows only that execution makes every
 reported number checkable. The text-only loops of the stopping calibration show,
 moreover, that the absence of execution does not by itself prevent unsupported
 statements: there, revisions credited the original analyses with procedures they never

contained. The loop’s weaknesses were equally clear. Panels raised more concerns than
an executor could address, repeatedly asked for data that did not exist, and never
declared an analysis acceptable; executors answered by expanding their scripts (three
of the four execution failures occurred in iterations that added large resampling
machinery) and, ~~and there is no mechanism for mid-loop human redirection beyond
the follow-up approval policy. Natural extensions include semantic merging and
convergence, multi-model ensemble execution that runs a task across several backends
and compares outputs, integration with laboratory-automation systems for wet-lab
validation, and richer human-in-the-loop control~~in two runs, by writing the verdict
into the code. Two of three runs reached the reference conclusion for TRRUST, but
none reported the positive part of the reference result, and one run designed itself into
non-identifiability. These observations argue for three additions that the evaluation
suggests but did not test: reviewers should be told what data are available so that
infeasible requests are recognized as such; conclusions, not only numbers, should be
traced to computed quantities, which would expose literal verdicts; and a human
should set the scope of an investigation and accept its conclusions. MI-Workbench
runs autonomously in the sense that a loop needs no intervention; our results indicate
that its conclusions should not be accepted without one.

Limitations. All model evaluations used one provider (OpenAI models through
the Codex CLI). The Claude Code and Gemini adapters are implemented and pass
offline tests against representative CLI output, but they were not run against live
models, so the results characterize one provider’s models and do not show whether
reviewers from other providers miss the same flaws or overlap as strongly across lenses;
merge quality and stopping behavior may also differ for other model families. All
benchmark conditions used the MI-specific reviewer prompts; a domain-agnostic
reviewer prompt was not tested, so the contribution of the domain-specific prompt
content to recall is not measured. Likewise, the optional generative and knowledge
roles (Table 2) were not evaluated. Codex adds its own harness instructions and a
user-level instruction file to every prompt. Both were constant across conditions, so
the comparisons between conditions do not depend on them; because the instruction
file asks the model to surface inconsistencies and to push back rather than agree, it
may raise the absolute numbers of critiques, flaws found and false alarms compared

with calls without it, which we did not test. The benchmark contains 18 synthetic write-ups (12 flawed, 6 clean) whose flaws are detectable from the text; flaws in real analyses can be subtler, and only planted flaws are scored, so unplanted concerns judged substantive may include real problems. The items were written and checked with Claude Opus 5.5, a model from a different provider than the reviewers and judges, so no evaluated model assessed text written by itself; the authoring model may nevertheless introduce stylistic regularities, including idiosyncratic phrasing of model facts, that make some flaws easier or harder to detect than in human-written analyses. Both judges are models from the same provider as the reviewers, and each is also one of the evaluated reviewer configurations; LLM judges can favor outputs similar to their own [58,59]. Neither judge favored its own model: the second judge was slightly more lenient than the primary judge for all three reviewer configurations, and least so for its own reviewers. The two judges also agreed closely, but neither observation excludes a bias shared by models of one provider. The merge evaluation scores only critiques matched to planted flaws, so over-merging of unrelated concerns is not measured; at the calibrated thresholds the lexical methods grouped more critiques across lenses than LLM adjudication did, the rule that keeps critiques of one reviewer call apart was not varied, and the optional sentence-embedding method was not evaluated. The stopping calibration uses twelve items, two configurations and a single judge, which is the same model as the executor and panel in one configuration and may therefore favor that configuration’s revisions; in the absence of data the executor could only revise text, and because its panels merged critiques at the uncalibrated lexical threshold, the executor received largely unmerged feedback and the grade trajectory is specific to that setting, whereas the fixed-budget outcomes and the persistence of high-severity critiques do not depend on the merge. The case study covers one question, one model, one tissue and three executed runs, with large differences between replicates; its evaluation was drafted with LLM assistance and compares conclusions that the runs reached with estimands different from ours. On macOS, verified execution relies on `sandbox-exec`, which Apple marks as deprecated, and only the `docker` backend caps memory; the confinement of the `docker` backend was checked only at the level of the container command, not against a running container. Coordination runs on a single node with SQLite, which was adequate for

the up to 20 concurrent runs we tested but is not designed for large deployments; the
real-backend test covered one item, one low-effort configuration and at most eight
concurrent panels, so provider rate limits were not probed.

~~In sum,~~

Conclusion

MI-Workbench ~~contributes a graph-based workflow DSL, a concurrent
consensus-review mechanism with a defined aggregation and gating semantics,
artifact-aware structured feedback, sandboxed verified execution, and a
provider-agnostic architecture, evaluated with both controlled orchestration
experiments and~~ provides an inspectable orchestration layer for critique-and-revision
loops on interpretability analyses of biological foundation models, with a ~~live-backend~~
case study. It ~~provides a foundation for scaling mechanistic interpretability of
biological foundation models beyond the capacity of individual researchers.
machine-readable critique contract, calibrated merging of duplicate critiques,
configurable stopping, verified execution in confined backends and a provenance record
of models, settings and outputs. Evaluated with OpenAI models on 18 synthetic
write-ups with planted flaws and in one executed case study, its reviewers caught most
planted statistical, confounding, causal and biological flaws, and its execution path
made every reported number traceable to code; whether reviewers from other
providers, or reviewers facing the subtler flaws of real analyses, perform as well
remains to be measured. The same evaluation shows where automated critique stops
being reliable; extra lenses add little coverage over extra samples, false premises about
the analyzed model go largely unnoticed, reviewers never declare an analysis
acceptable, and loops left to run accumulate new errors. The benchmark, the
calibration data and the code are released so that these limits can be measured again
as models and providers change.~~

Supporting information

S1 Text. System details. Data model, run lifecycle and resumption, retry and error handling, output cleaning, adapter invocations, knowledge extraction, version control, reproducibility packages and codebase statistics.

S1 Fig. MI-Workbench architecture diagram (high resolution)Orchestration overhead and real-backend concurrency.

Full-resolution version of Fig 1, provided as a standalone vector source (Fig1.tex) for reproduction (A) Control-plane overhead per iteration (median and 95th percentile) against the number of concurrent runs, measured through the production runner with a real database and artifact writing, for an instantaneous and a 1-s mock backend on an internal and an external drive. Measurements were made while other jobs ran on the same workstation. (B) Throughput of simultaneous three-lens panels with a real backend (gpt-5.6-luna, low effort) against the number of concurrent panels, with linear scaling for reference.

S1 FileTable. Source-code-and-reproducibility-packageRun register. The complete MI-Workbench implementation (backend, CLI, frontend, prompt templates, loop presets, Every LLM-based experiment with provider, CLI version, model, reasoning-effort setting, decoding, tool settings, numbers of calls and attempts, input, cached-input and output tokens, wall time and data location.

S2 Table. Full results of the benchmark, merge evaluation and stopping calibration. Recall by condition, configuration and judge; planned contrasts; severity-aware recall; recall by flaw family, type and individual flaw; discrimination; high and critical critiques on clean items; composition of unmatched critiques; cost; judge agreement; merge evaluation; sensitivity analysis with TF-IDF merging at $\tau = 0.075$; stopping-rule outcomes and per-revision trajectories. In the stopping-rule sheet, fixed_k1 to fixed_k5 denote budgets of one to five revisions (the platform default is fixed_k4), default_any_of_3 denotes the adaptive rule that stops when any of the three signals holds, and suffixes @window2, @sim0.8 and test-suite) is available at . Reproducibility packages for the case study runs are generated by the repropack module and bundle all artifacts, resolved prompts, environment metadata, and git provenance@sim0.95 the sensitivity variants.

S1 File. **Planted-flaw benchmark.** The 18 analysis write-ups, ground truth with
detection criteria, independent verification notes, the frozen analysis plan and
manifest, the judge rubric and the reviewer prompts.

~~S2 File. **Evaluation harness and results.**~~Data and analysis code for the
benchmark, merge evaluation and stopping calibration. ~~The orchestration~~
~~experiments (panel ablation, reviewer-lens overlap, convergence profile, and~~
~~concurrent-run load benchmark) are implemented in the~~
~~backend/analysis/experiments.py module and reproduced by~~
~~scripts/run_experiments.py, which writes the machine-readable result~~
~~consumed by Table ?? and Fig 2. The live-backend corroboration is reproduced by~~
~~scripts/real_consensus_probe.py.~~Raw reviewer outputs, judge labels,
adjudications, loop trajectories and state judgments, analysis scripts and processed
results.

S3 File. **Case-study materials.** Data-kit construction and independent-analysis
code with results and provenance, the task texts, the four agent runs (sol_A, sol_B,
gpt55 and sol_planonly, corresponding to runs 1–3 and the control in Table 5; all
iterations, execution outputs and run metadata), the number trace, and a synthesis of
the runs (per-iteration results, the panel-identified defects with the code lines and
outputs used to confirm them and their fix status, and the panel’s incorrect critiques)
together with notes from its independent verification against the source files.

Acknowledgments

The author thanks the developers of the open-source tools on which MI-Workbench is
built, including FastAPI, Pydantic, and the Claude Code, OpenAI Codex , and Google
Gemini command-line interfaces, and the authors of Geneformer, Tabula Sapiens,
TRRUST and DoRothEA for making their models and data available.

References

1. Cui H, Wang C, Maan H, Pang K, Luo F, Duan N, et al. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*. 2024;21(8):1470–1480.
2. Theodoris CV, Xiao L, Chopra A, Chaffin MD, Al Sayed ZR, Hill MC, et al. Transfer learning enables predictions in network biology. *Nature*. 2023;618(7965):616–624.
3. Yang F, Wang W, Wang F, Fang Y, Tang D, Huang J, et al. scBERT as a large-scale pretrained deep language model for cell type annotation of single-cell RNA-seq data. *Nature Machine Intelligence*. 2022;4(10):852–866.
4. Olah C, Cammarata N, Schubert L, Goh G, Petrov M, Carter S. Zoom in: an introduction to circuits. *Distill*. 2020;5(3):e00024.001.
5. Elhage N, Hume T, Olsson C, Schiefer N, Henighan T, Kravec S, et al. Toy models of superposition; 2022. Transformer Circuits Thread. Available from: https://transformer-circuits.pub/2022/toy_model/index.html.
6. Conmy A, Mavor-Parker AN, Lynch A, Heimersheim S, Garriga-Alonso A. Towards automated circuit discovery for mechanistic interpretability. *Advances in Neural Information Processing Systems*. 2023;36.
7. Pratapa A, Jalihal AP, Law JN, Bharadwaj A, Murali TM. Benchmarking algorithms for gene regulatory network inference from single-cell transcriptomic data. *Nature Methods*. 2020;17(2):147–154.
8. Jain S, Wallace BC. Attention is not Explanation. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics; 2019. p. 3543–3556.
9. Wiegrefe S, Pinter Y. Attention is not not Explanation. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics; 2019. p. 11–20.

10. Serrano S, Smith NA. Is Attention Interpretable? In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics; 2019. p. 2931–2951.
11. Kedzierska KZ, Crawford L, Amini AP, Lu AX. Zero-shot evaluation reveals limitations of single-cell foundation models. *Genome Biology*. 2025;26:101. doi:10.1186/s13059-025-03574-x.
12. Boiarsky R, Singh N, Buendia A, Getz G, Sontag D. A Deep Dive into Single-Cell RNA Sequencing Foundation Models. *bioRxiv*. 2023;doi:10.1101/2023.10.19.563100.
13. Kendiukhov I. Systematic evaluation of single-cell foundation model interpretability: attention-derived edge scores add no incremental value over gene-level features for perturbation-target prediction. *BMC Genomics*. 2026;27:634. doi:10.1186/s12864-026-12965-8.
14. Farquhar S, Kossen J, Kuhn L, Gal Y. Detecting hallucinations in large language models using semantic entropy. *Nature*. 2024;630(8017):625–630. doi:10.1038/s41586-024-07421-0.
15. Beel J, Kan MY, Baumgart M. Evaluating Sakana’s AI Scientist: Bold Claims, Mixed Results, and a Promising Future? *ACM SIGIR Forum*. 2025;59(1):1–20. doi:10.1145/3769733.3769747.
16. Lu C, Lu C, Lange RT, Yamada Y, Hu S, Foerster J, et al. Towards end-to-end automation of AI research. *Nature*. 2026;651(8107):914–919. doi:10.1038/s41586-026-10265-5.
17. Schmidgall S, Su Y, Wang Z, Sun X, Wu J, Yu X, et al. Agent Laboratory: Using LLM Agents as Research Assistants. In: Findings of the Association for Computational Linguistics: EMNLP 2025. Suzhou, China: Association for Computational Linguistics; 2025. p. 5977–6043.
18. Gottweis J, Weng WH, Daryin A, Tu T, Sirkovic P, Myaskovsky A, et al. Accelerating scientific discovery with Co-Scientist. *Nature*. 2026;655(8122):487–496. doi:10.1038/s41586-026-10644-y.

19. Swanson K, Wu W, Bulaong NL, Pak JE, Zou J. The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies. *Nature*. 2025;646(8085):716–723. doi:10.1038/s41586-025-09442-9.
20. Xiao Y, Liu J, Zheng Y, Jiao S, Hao J, Xie X, et al. CellAgent: LLM-Driven Multi-Agent Framework for Natural Language-Based Single-Cell Analysis. In: *The Fourteenth International Conference on Learning Representations (ICLR)*; 2026. Available from: <https://openreview.net/forum?id=BsA2GNkJhz>.
21. Alber S, Chen B, Sun E, Isakova A, Wilk AJ, Zou J. CellVoyager: AI CompBio agent generates new insights by autonomously analyzing biological data. *Nature Methods*. 2026;23(4):749–759. doi:10.1038/s41592-026-03029-6.
22. Wang Q, Wang Z, Su Y, Tong H, Song Y. Rethinking the Bounds of LLM Reasoning: Are Multi-Agent Discussions the Key? In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics; 2024. p. 6106–6131.
23. Smit AP, Grinsztajn N, Duckworth P, Barrett TD, Pretorius A. Should we be going MAD? A Look at Multi-Agent Debate Strategies for LLMs. In: *Proceedings of the 41st International Conference on Machine Learning*. vol. 235 of *Proceedings of Machine Learning Research*. PMLR; 2024. p. 45883–45905.
24. Lu C, Lu C, Lange RT, Foerster J, Clune J, Ha D. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery; 2024. arXiv:2408.06292. Available from: <https://arxiv.org/abs/2408.06292>.
25. Yamada Y, Lange RT, Lu C, Hu S, Lu C, Foerster J, et al. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search; 2025. arXiv:2504.08066. Available from: <https://arxiv.org/abs/2504.08066>.
26. Gottweis J, Weng WH, Daryin A, Tu T, Palepu A, Sirkovic P, et al. Towards an AI co-scientist; 2025. arXiv:2502.18864. Available from: <https://arxiv.org/abs/2502.18864>.

27. Ghareeb AE, Chang B, Mitchener L, Yiu A, Szostkiewicz CJ, Shved D, et al. A multi-agent system for automating scientific discovery. *Nature*. 2026;655(8122):497–505. doi:10.1038/s41586-026-10652-y.
28. Mitchener L, Yiu A, Chang B, Bourdenx M, Nadolski T, Sulovari A, et al. Kosmos: An AI Scientist for Autonomous Discovery; 2025. arXiv:2511.02824. Available from: <https://arxiv.org/abs/2511.02824>.
29. Bran AM, Cox S, Schilter O, Baldassari C, White AD, Schwaller P. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*. 2024;6(5):525–535.
30. Boiko DA, MacKnight R, Kline B, Gomes G. Autonomous chemical research with large language models. *Nature*. 2023;624(7992):570–578.
31. Ghafarollahi A, Buehler MJ. SciAgents: automating scientific discovery through bioinspired multi-agent intelligent graph reasoning. *Advanced Materials*. 2025;37(22):2413523.
32. Xiao Y, Liu J, Zheng Y, Xie X, Hao J, Li M, et al. CellAgent: An LLM-driven Multi-Agent Framework for Automated Single-cell Data Analysis; 2024. arXiv:2407.09811. Available from: <https://arxiv.org/abs/2407.09811>.
33. Huang K, Zhang S, Wang H, Qu Y, Lu Y, Li R, et al. Autonomous biomedical research with an artificial intelligence agent. *Science*. 2026;393(6813):eadz4351. doi:10.1126/science.adz4351.
34. Zhou J, Zhang B, Li G, Chen X, Li H, Xu X, et al. An AI Agent for Fully Automated Multi-Omic Analyses. *Advanced Science*. 2024;11(44):2407094. doi:10.1002/advs.202407094.
35. Roohani YH, Lee AH, Huang Q, Vora J, Steinhart Z, Huang K, et al. BioDiscoveryAgent: An AI Agent for Designing Genetic Perturbation Experiments. In: *The Thirteenth International Conference on Learning Representations (ICLR)*; 2025. Available from: <https://arxiv.org/abs/2405.17631>.

36. Qu Y, Huang K, Yin M, Zhan K, Liu D, Yin D, et al. CRISPR-GPT for agentic automation of gene-editing experiments. *Nature Biomedical Engineering*. 2026;10(2):245–258. doi:10.1038/s41551-025-01463-z.
37. Wang Z, Jin Q, Wei CH, Tian S, Lai PT, Zhu Q, et al. GeneAgent: self-verification language agent for gene-set analysis using domain databases. *Nature Methods*. 2025;22(8):1677–1685. doi:10.1038/s41592-025-02748-6.
38. Xie E, Cheng L, Shireman J, Cai Y, Liu J, Mohanty C, et al. CASSIA: a multi-agent large language model for automated and interpretable cell annotation. *Nature Communications*. 2026;17:389. doi:10.1038/s41467-025-67084-x.
39. Bills S, Cammarata N, Mossing D, Tillman H, Gao L, Goh G, et al. Language models can explain neurons in language models; 2023. OpenAI. Available from: <https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html>.
40. Paulo GS, Mallen AT, Juang C, Belrose N. Automatically Interpreting Millions of Features in Large Language Models. In: *Proceedings of the 42nd International Conference on Machine Learning*. vol. 267 of *Proceedings of Machine Learning Research*. PMLR; 2025. p. 48393–48421.
41. Rott Shaham T, Schwettmann S, Wang F, Rajaram A, Hernandez E, Andreas J, et al. A Multimodal Automated Interpretability Agent. In: *Proceedings of the 41st International Conference on Machine Learning*. vol. 235 of *Proceedings of Machine Learning Research*. PMLR; 2024. p. 44293–44321.
42. Simon E, Zou J. InterPLM: discovering interpretable features in protein language models via sparse autoencoders. *Nature Methods*. 2025;22(10):2107–2117. doi:10.1038/s41592-025-02836-7.
43. Gujral O, Bafna M, Alm E, Berger B. Sparse autoencoders uncover biologically interpretable features in protein language model representations. *Proceedings of the National Academy of Sciences*. 2025;122(34):e2506316122. doi:10.1073/pnas.2506316122.

44. Liu T, Li K, Wang Y, Li H, Zhao H. Evaluating the Utilities of Foundation Models in Single-Cell Data Analysis. *Advanced Science*. 2026;13(27):e14490. doi:10.1002/advs.202514490.
45. Kalfon J, Samaran J, Peyré G, Cantini L. scPRINT: pre-training on 50 million cells allows robust gene network predictions. *Nature Communications*. 2025;16:3607. doi:10.1038/s41467-025-58699-1.
46. Ye Z, Yang N, Yang X, Mao X, Tang C. Benchmarking Static Gene Regulatory Network Reconstruction and Dynamic Transition Probing in Single-Cell Foundation Models. *bioRxiv*. 2026;doi:10.64898/2026.05.17.725083.
47. Pedrocchi F, Barkmann F, Joudaki A, Boeva V. Sparse Autoencoders Reveal Interpretable Features in Single-Cell Foundation Models. *bioRxiv*. 2025;doi:10.1101/2025.10.22.681631.
48. Li G, Hammoud HAAK, Itani H, Khizbullin D, Ghanem B. CAMEL: communicative agents for “mind” exploration of large language model society. *Advances in Neural Information Processing Systems*. 2023;36.
49. Wu Q, Bansal G, Zhang J, Wu Y, Li B, Zhu E, et al. AutoGen: enabling next-gen LLM applications via multi-agent conversation; 2023. arXiv:2308.08155. Available from: <https://arxiv.org/abs/2308.08155>.
50. Qian C, Liu W, Liu H, Chen N, Dang Y, Li J, et al. ChatDev: communicative agents for software development. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*; 2024. p. 15174–15186.
51. Hong S, Zhuge M, Chen J, Zheng X, Cheng Y, Zhang C, et al. MetaGPT: meta programming for a multi-agent collaborative framework. In: *The Twelfth International Conference on Learning Representations (ICLR)*; 2024.
52. Du Y, Li S, Torralba A, Tenenbaum JB, Mordatch I. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In: *Proceedings of the 41st International Conference on Machine Learning*. vol. 235 of *Proceedings of Machine Learning Research*. PMLR; 2024. p. 11733–11763.

53. D’Arcy M, Hope T, Birnbaum L, Downey D. MARG: Multi-Agent Review Generation for Scientific Papers; 2024. arXiv:2401.04259. Available from: <https://arxiv.org/abs/2401.04259>.
54. Jin Y, Zhao Q, Wang Y, Chen H, Zhu K, Xiao Y, et al. AgentReview: Exploring Peer Review Dynamics with LLM Agents. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics; 2024. p. 1208–1226.
55. Wang X, Wei J, Schuurmans D, Le Q, Chi E, Narang S, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In: The Eleventh International Conference on Learning Representations (ICLR); 2023. Available from: <https://arxiv.org/abs/2203.11171>.
56. Li J, Zhang Q, Yu Y, Fu Q, Ye D. More Agents Is All You Need. Transactions on Machine Learning Research. 2024;.
57. Cemri M, Pan MZ, Yang S, Agrawal LA, Chopra B, Tiwari R, et al. Why Do Multi-Agent LLM Systems Fail? In: Advances in Neural Information Processing Systems 38 (Datasets and Benchmarks Track); 2025.
58. Zheng L, Chiang WL, Sheng Y, Zhuang S, Wu Z, Zhuang Y, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In: Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track); 2023. p. 46595–46623.
59. Wang P, Li L, Chen L, Cai Z, Zhu D, Lin B, et al. Large Language Models are not Fair Evaluators. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics; 2024. p. 9440–9450.
60. Kuhn L, Gal Y, Farquhar S. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In: The Eleventh International Conference on Learning Representations (ICLR); 2023. Available from: <https://arxiv.org/abs/2302.09664>.

61. Reimers N, Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Association for Computational Linguistics; 2019. p. 3980–3990.
62. Afgan E, Baker D, Batut B, van den Beek M, Bouvier D, Čech M, et al. The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update. *Nucleic Acids Research*. 2018;46(W1):W537–W544.
63. Di Tommaso P, Chatzou M, Floden EW, Barja PP, Palumbo E, Notredame C. Nextflow enables reproducible computational workflows. *Nature Biotechnology*. 2017;35(4):316–319.
64. Mölder F, Jablonski KP, Letcher B, Hall MB, Tomkins-Tinch CH, Sochat V, et al. Sustainable data analysis with Snakemake. *F1000Research*. 2021;10:33.
65. Cohen J. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*. 1960;20(1):37–46. doi:10.1177/001316446002000104.
66. Holm S. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*. 1979;6(2):65–70.
67. The Tabula Sapiens Consortium. The Tabula Sapiens: a multiple-organ, single-cell transcriptomic atlas of humans. *Science*. 2022;376(6594):eabl4896. doi:10.1126/science.abl4896.
68. Han H, Cho JW, Lee S, Yun A, Kim H, Bae D, et al. TRRUST v2: an expanded reference database of human and mouse transcriptional regulatory interactions. *Nucleic Acids Research*. 2018;46(D1):D380–D386. doi:10.1093/nar/gkx1013.
69. Garcia-Alonso L, Holland CH, Ibrahim MM, Turei D, Saez-Rodriguez J. Benchmark and integration of resources for the estimation of human transcription factor activities. *Genome Research*. 2019;29(8):1363–1375. doi:10.1101/gr.240663.118.
70. Benjamini Y, Hochberg Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society*:

- Series B (Methodological). 1995;57(1):289–300.
doi:10.1111/j.2517-6161.1995.tb02031.x.
71. Pydantic. Pydantic: data validation using Python type hints; 2026.
<https://docs.pydantic.dev/>.
72. Kendiukhov I. MI-Workbench; 2026. Zenodo. Available from:
<https://doi.org/10.5281/zenodo.23097814>.
73. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al.
Attention is All you Need. In: Advances in Neural Information Processing
Systems. vol. 30. Curran Associates, Inc.; 2017. p. 5998–6008. Available from:
[https://proceedings.neurips.cc/paper_files/paper/2017/file/
3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf).
74. Tsai YHH, Bai S, Yamada M, Morency LP, Salakhutdinov R. Transformer
Dissection: An Unified Understanding for Transformer’s Attention via the Lens of
Kernel. In: Proceedings of the 2019 Conference on Empirical Methods in Natural
Language Processing and the 9th International Joint Conference on Natural
Language Processing (EMNLP-IJCNLP). Association for Computational
Linguistics; 2019. p. 4343–4352.